Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



Кафедра 09.02.07_Информационные системы и программирование

ПРАКТИЧЕСКАЯ РАБОТА № 6

Дисциплина(модуль)/предмет МДК 03.01 Внедрение и поддержка компьютерных систем

Тема: Сопровождение ИС «Умный кампус» на этапе эксплуатации по CALS-технологиям: управление обновлениями (Patch Management) и проверка готовности АРМ оператора

ВАРИАНТ № 10 «Тестер Health Check после обновления» (API: https://api.ipify.org?format=json)

Студент (ка)/учащийся: Леонов Тимофей Андреевич /_____ /
Ф. И. О. подпись

Группа: И14-23-9

Руководитель: Чудинова А.А /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «24» октября 2026г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 3
1 ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА..... 5
2 РЕГЛАМЕНТ УПРАВЛЕНИЯ ОБНОВЛЕНИЯМИ..... 7
2.1 Схема доставки обновлений..... 7
2.2 Структурная схема регламента..... 9
2.3 Резервное копирование локальных данных..... 11
2.4 План отката..... 12
3 КОНСОЛЬНАЯ УТИЛИТА ПРОВЕРКИ ГОТОВНОСТИ АРМ..... 13
3.1 Структура проекта..... 13
3.2 Цепочка проверок..... 15
3.3 Обработка исключений..... 16
3.4 Акт готовности АРМ..... 18
4 ЭКСПЛУАТАЦИОННАЯ ДОКУМЕНТАЦИЯ..... 19
4.1 Состав документации..... 19
4.2 Примеры оформления документов..... 20
5 SANITY-ТЕСТ И ИСПЫТАНИЯ НА СТЕНДЕ..... 24
5.1 Стенд опытной эксплуатации..... 24
5.2 Штатное обновление..... 25
5.3 Сбой обновления и автоматический откат..... 26
5.4 Повреждённый пакет и сетевой таймаут..... 27
5.5 Ручной откат..... 28
5.6 Акт готовности..... 29
5.7 Автоматизированный sanity-тест..... 29
ЗАКЛЮЧЕНИЕ..... 33
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ..... 35
ПРИЛОЖЕНИЕ А Исходный код утилиты Campus.Arm.Updater..... 37

ВВЕДЕНИЕ

Информационная система «Умный кампус» включает клиентский модуль «АРМ оператора», который работает на распределённых рабочих местах: контрольно-пропускных пунктах, в библиотеке и деканате. Каждый АРМ хранит локальные данные (настройки, кэш справочников, журнал проходов в базе SQLite) и обращается к центральному API. Наибольшая часть жизненного цикла такой системы приходится на этап эксплуатации, и именно на нём выпускаются исправления и новые версии клиента.

Актуальность темы определяется тем, что ошибка при обновлении АРМ КПП сразу останавливает пропуск людей на территорию, а потеря локального журнала проходов означает потерю данных, которые ещё не переданы на сервер. Ручная установка обновлений без резервного копирования, проверки работоспособности и плана отката приводит к длительным простоям. CALS-технологии предписывают сопровождать изделие на всех стадиях жизненного цикла на основе единых электронных данных [1, 2], поэтому каждое обновление должно оставлять проверяемый электронный след: журнал, акт готовности и запись в журнале изменений.

Объект исследования – процесс сопровождения клиентского модуля «АРМ оператора» ИС «Умный кампус» на этапе эксплуатации. Предмет исследования – управление обновлениями (Patch Management) и проверка готовности АРМ после обновления.

Новизна работы состоит в том, что проверка готовности (Health Check) встроена в сам процесс обновления: при провале критичной проверки утилита автоматически возвращает предыдущую версию и данные, а результат каждой операции оформляется актом готовности с замером задержек проверок.

Практическая значимость: разработанные регламент, утилита и эксплуатационная документация могут применяться администратором для

планового обновления АРМ без остановки работы оператора и с гарантированной возможностью отката.

Цель работы – организовать управление обновлениями АРМ оператора на этапе эксплуатации и автоматизировать проверку готовности АРМ после обновления по варианту «Тестер Health Check после обновления».

Для достижения цели поставлены задачи:

– разработать регламент доставки обновлений по схемам Canary и Blue-Green, резервного копирования локальных данных и отката;

– разработать на C# (.NET 8) консольную утилиту, выполняющую цепочку проверок базы данных, API и считывателя с журналированием задержек и формированием акта готовности;

– обеспечить обработку сетевых таймаутов и ошибок файлового ввода-вывода;

– подготовить эксплуатационную документацию по принципу Docs-as-Code в соответствии с ГОСТ 19;

– провести sanity-тест: имитацию обновления с проверкой целостности данных и формированием акта.

Методы работы: анализ требований к сопровождению, моделирование процессов с помощью блок-схем в draw.io, объектно-ориентированное программирование, автоматизированное тестирование и испытания на стенде опытной эксплуатации.

1 ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА

Объект сопровождения – клиентский модуль «АРМ оператора» ИС «Умный кампус»: настольное приложение, установленное на распределённых рабочих местах (КПП, библиотека, деканат). АРМ хранит локальные данные – файл настроек appsettings.json, кэш справочников и локальную БД SQLite с журналом проходов и очередью синхронизации, работает со считывателем карт и обращается к центральному API.

Задание варианта № 10 – «Тестер Health Check после обновления»: утилита выполняет цепочку проверок (БД, API, считыватель), логирует задержки (Latency) и формирует акт готовности АРМ. Внешний API для проверки сетевой доступности – https://api.ipify.org?format=json, он возвращает внешний IP-адрес АРМ в формате JSON [12].

Утилита Campus.Arm.Updater разработана на C# (.NET 8, консольное приложение) в среде Microsoft Visual Studio. Кроме проверок готовности она реализует весь регламент обновления, поэтому проверка выполняется в составе процесса обновления. Состав работы приведён в таблице 1.

Таблица 1 – Состав работы

Этап задания	Результат (папка homework-6 репозитория)
Регламент Patch Management	UPDATE_GUIDE.md, схемы в разделе 2 отчёта
Утилита C# по варианту	Campus.Arm.Updater.sln, src/Campus.Arm.Updater (14 файлов .cs)
Docs-as-Code	OPERATOR_MANUAL.md, UPDATE_GUIDE.md, CHANGELOG.md
Sanity-тест	tests/Campus.Arm.Updater.Tests/SanityTests.cs (8 тестов), примеры актов docs/examples
Непрерывная интеграция	workflow homework-6 CI – сборка и тесты на каждый push

Источник: составлено автором

Все файлы размещены в репозитории https://github.com/kotokotru/smart-campus в папке homework-6 (рисунок 1).

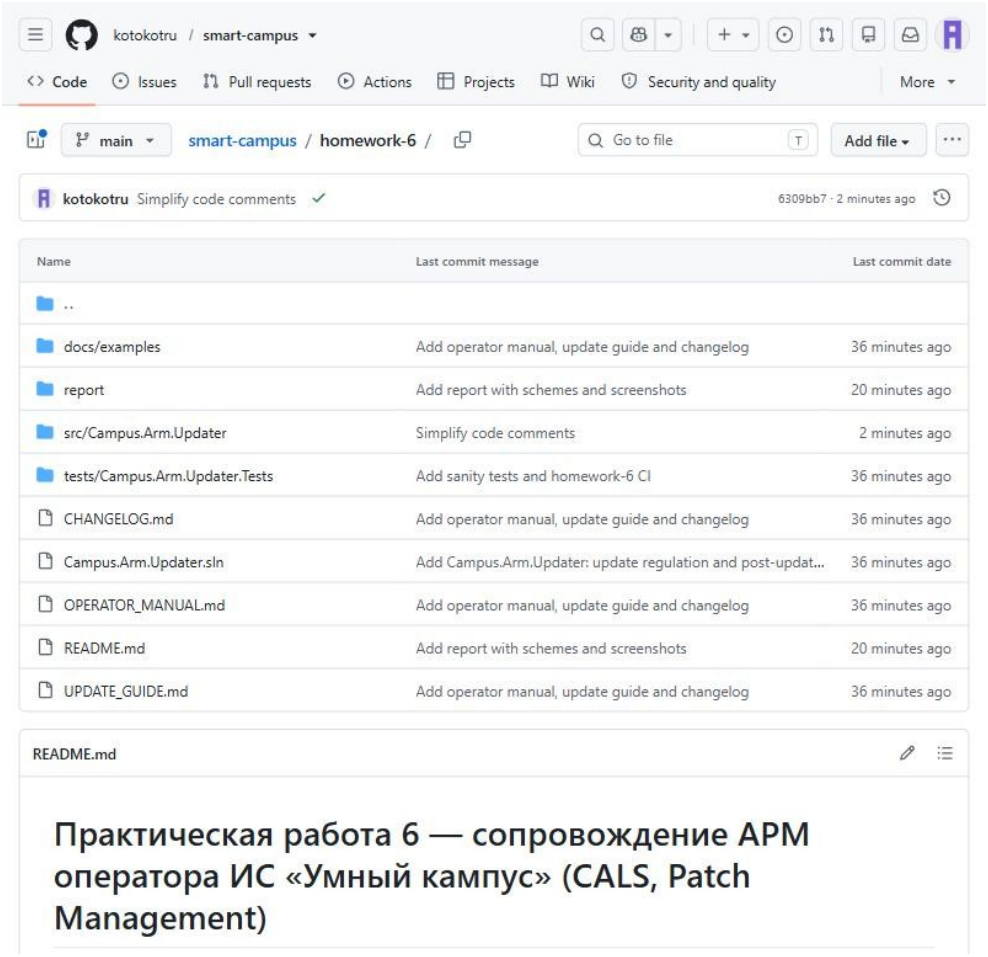


Рисунок 1 – Папка homework-6 в репозитории на GitHub

2 РЕГЛАМЕНТ УПРАВЛЕНИЯ ОБНОВЛЕНИЯМИ

2.1 Схема доставки обновлений

Регламент определяет, как новая версия АРМ доставляется с центрального сервера на рабочие места, как сохраняются локальные данные и как выполняется откат при сбое. Используются две стратегии одновременно (рисунок 2):

– Canary Deployment – кольца развёртывания: новый релиз публикуется в манифесте сервера обновлений только для кольца canary (два пилотных АРМ КПП); после трёх рабочих дней опытной эксплуатации и актов «ГОТОВ» администратор добавляет кольцо stable, и релиз становится доступен остальным АРМ; релиз, на котором пилотный АРМ откатился, в stable не выпускается [3];

– Blue-Green на каждом АРМ: клиент установлен в два слота slots/blue и slots/green, файл active-slot.txt указывает, какой слот запускается; новая версия ставится в неактивный слот, не мешая работе оператора, и становится активной атомарной заменой этого файла, а откат – обратное переключение за доли секунды [4].

Схема разработана в редакторе draw.io, исходный файл delivery.drawio хранится в папке report/schemes репозитория.

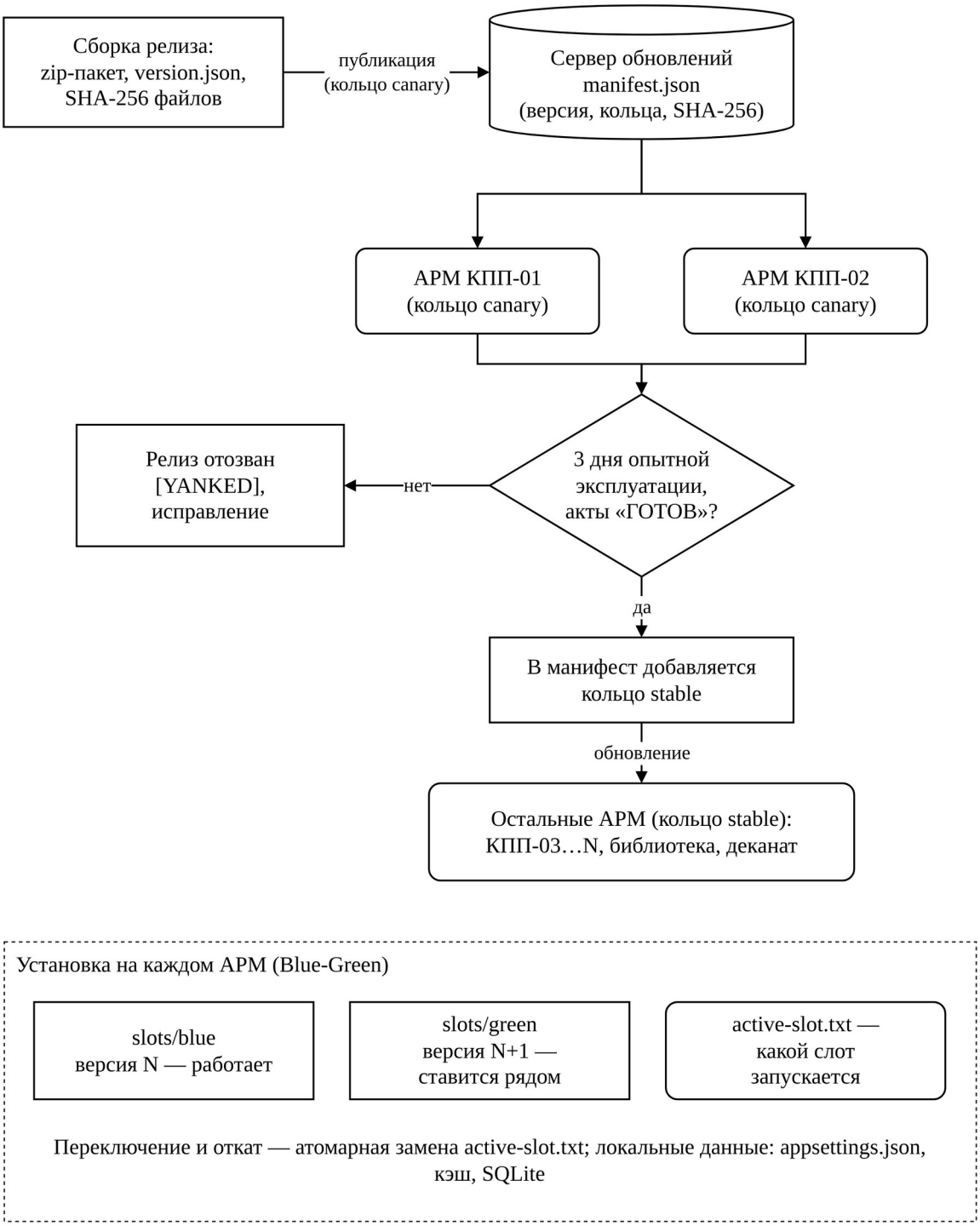


Рисунок 2 – Схема доставки обновлений

2.2 Структурная схема регламента

Последовательность шагов обновления с точками принятия решений и ветками отката показана на блок-схеме (рисунок 3, исходный файл regulation.drawio) и в таблице 2.

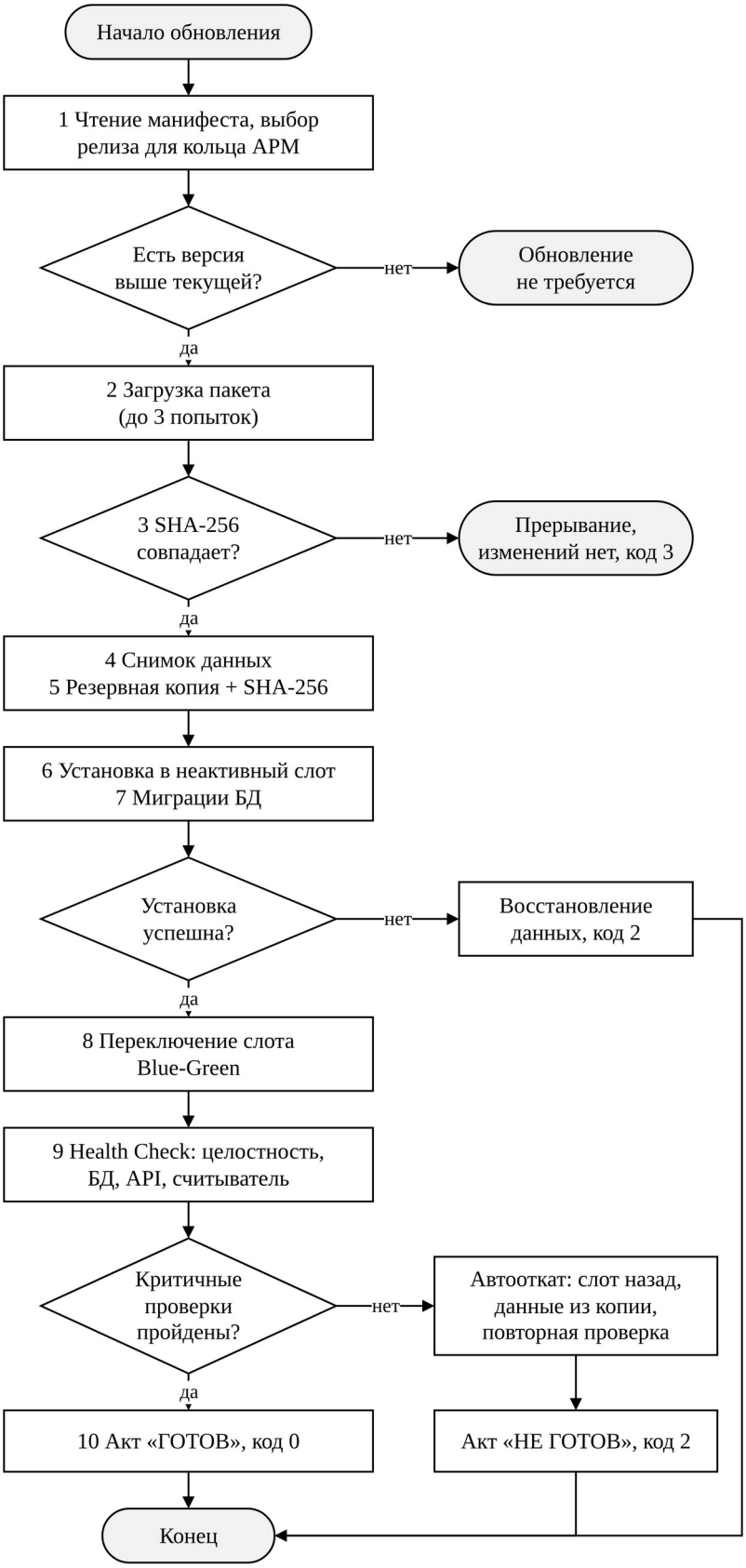


Рисунок 3 – Блок-схема регламента обновления АРМ оператора

Таблица 2 – Шаги регламента обновления

№	Шаг	Критерий перехода	При ошибке
1	Чтение манифеста, выбор релиза для кольца АРМ	есть версия выше текущей	код 3, изменений нет
2	Загрузка пакета (3 попытки при ошибке ввода-вывода)	файл получен	код 3, изменений нет
3	Проверка SHA-256 пакета по манифесту	сумма совпадает	код 3, изменений нет
4	Снимок числа записей в ключевых таблицах	–	–
5	Резервная копия: appsettings.json, кэш, SQLite	копия проверена по SHA-256	обновление запрещено
6	Установка в неактивный слот, проверка файлов	все SHA-256 совпадают	восстановление данных, код 2
7	SQL-миграции, каждая в своей транзакции	схема БД обновлена	восстановление данных, код 2
8	Переключение слота Blue-Green	–	–
9	Health Check: целостность, БД, API, считыватель	нет проваленных критичных проверок	автоматический откат, код 2
10	Акт готовности АРМ	акт «ГОТОВ»	акт «НЕ ГОТОВ» разработчикам

Источник: составлено автором

Окно обновления АРМ КПП – с 00:30 до 05:30, когда проход через КПП минимален; АРМ библиотеки и деканата обновляются после окончания рабочего дня.

2.3 Резервное копирование локальных данных

Копия создаётся автоматически перед каждым обновлением в каталоге BackupRoot с именем, содержащим дату, время и номер версии (таблица 3). В каталог копии записывается файл backup.json с SHA-256 каждого файла; копия проверяется сразу после создания и повторно перед восстановлением. База SQLite копируется встроенным механизмом онлайн-копирования [11]. Хранятся последние пять копий.

Таблица 3 – Резервируемые данные АРМ

Данные	Способ копирования	Назначение
data/appsettings.json	копирование файла	настройки АРМ: номер, зона турникета, режим считывателя
data/cache	копирование каталога	кэш справочников операторов и правил доступа
data/campus-local.db	онлайн-копия средствами SQLite (BackupDatabase)	журнал проходов и очередь синхронизации

Источник: составлено автором

2.4 План отката

Критерии отката: провал любой критичной проверки Health Check после переключения слота, ошибка распаковки пакета или миграции БД, подтверждённые проверкой жалобы оператора. Предусмотрены три сценария:

1) автоматический откат выполняет утилита: активный слот возвращается на предыдущую версию, данные восстанавливаются из резервной копии, слот с непроверенной версией очищается, выполняется повторный Health Check и формируется акт «НЕ ГОТОВ … выполнен откат»;

2) ручной откат выполняет администратор командой rollback: после синхронизации событий с сервером утилита находит последнюю резервную копию версии ниже текущей, при необходимости заново устанавливает эту версию с сервера обновлений, переключает слот, восстанавливает данные, проверяет АРМ и формирует акт;

3) при повреждённом пакете или недоступном сервере обновление прерывается до установки, на АРМ ничего не меняется и откат не требуется.

Откат невозможен, если на АРМ нет резервной копии версии ниже текущей или пакет этой версии удалён с сервера, поэтому пакеты версий, выпущенных в stable, хранятся на сервере не менее шести месяцев.

3 КОНСОЛЬНАЯ УТИЛИТА ПРОВЕРКИ ГОТОВНОСТИ АРМ

3.1 Структура проекта

Решение Campus.Arm.Updater.sln содержит консольное приложение и проект тестов. Приложение разделено на слои (таблица 4); полный исходный код с комментариями приведён в приложении А.

Таблица 4 – Структура проекта Campus.Arm.Updater

Файл	Назначение
Program.cs	разбор команд status, check, update, healthcheck, rollback, demo-init; коды завершения
Config/UpdaterOptions.cs	настройки updater.json: АРМ, кольцо, каталоги, параметры проверок
Update/UpdateOrchestrator.cs	регламент обновления, автоматический и ручной откат
Update/BackupService.cs	создание, проверка, ротация и восстановление резервных копий
Update/BlueGreenInstaller.cs	установка пакета в неактивный слот, SQL-миграции
Update/ArmLayout.cs	структура каталогов АРМ, переключение слота, манифест
Health/Checks.cs	четыре проверки: целостность, БД, API, считыватель
Health/HealthCheckRunner.cs	выполнение цепочки, замер задержки, обработка исключений
Reporting/ReadinessAct.cs	акт готовности АРМ и итоговая таблица в консоли
Infrastructure	журнал, SHA-256, повтор файловых операций, доступ к SQLite
Demo/DemoEnvironment.cs	стенд опытной эксплуатации: АРМ 2.4.0 и сервер с релизами 2.5.0–2.6.0

Источник: составлено автором

Решение открыто и собрано в Microsoft Visual Studio без ошибок и предупреждений (рисунок 4); фрагмент класса ApiCheck с комментариями показан на рисунке 5.

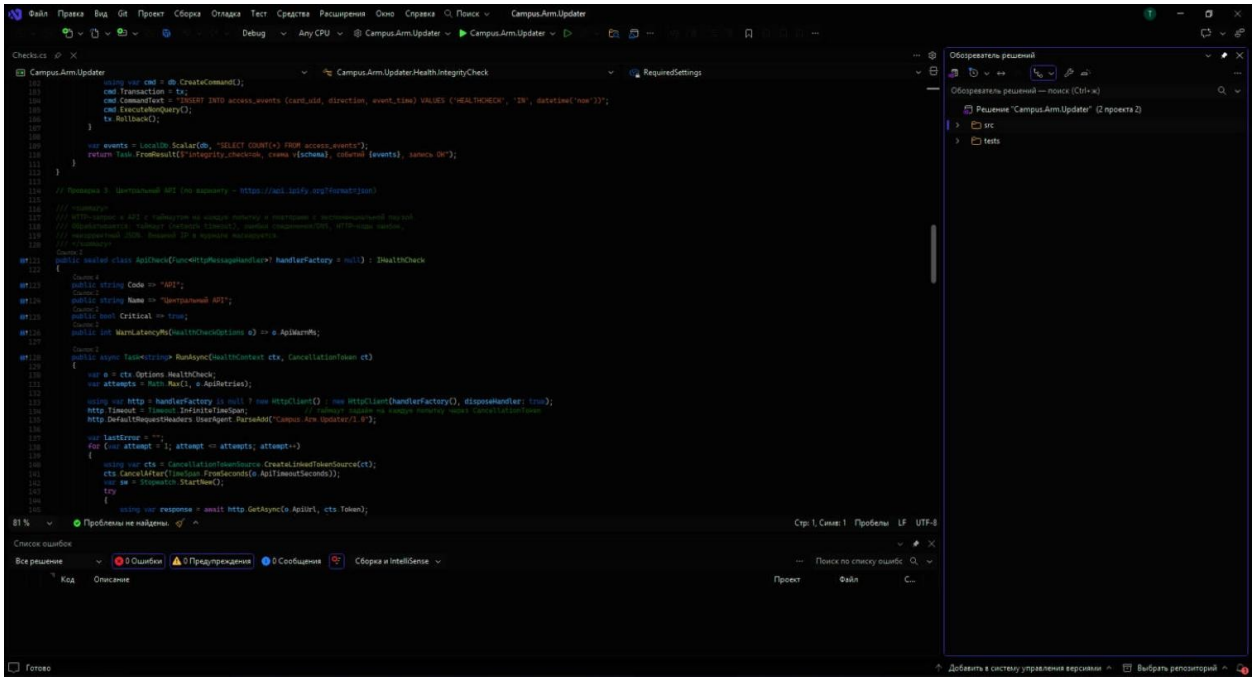


Рисунок 4 – Решение Campus.Arm.Updater в Microsoft Visual Studio

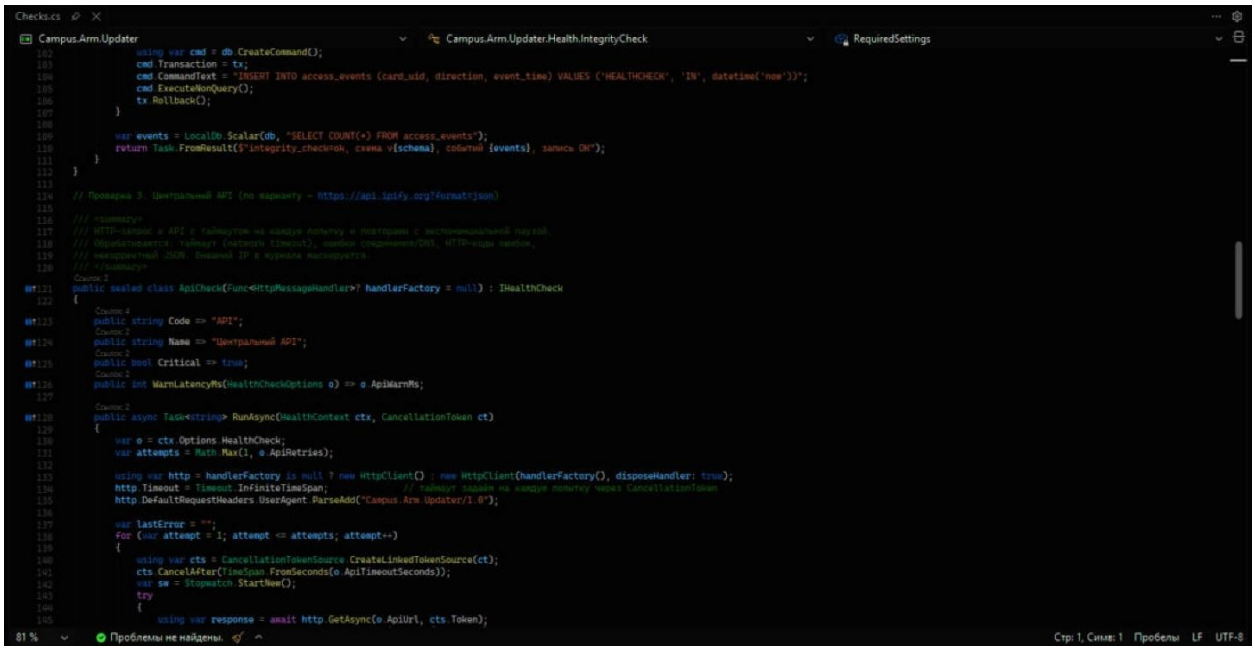


Рисунок 5 – Класс ApiCheck в редакторе Visual Studio

3.2 Цепочка проверок

Проверки выполняются строго по порядку; задержка каждой измеряется таймером Stopwatch и пишется в журнал. Превышение порога задержки даёт статус «ПРЕДУПРЕЖДЕНИЕ», который не блокирует готовность (таблица 5).

Таблица 5 – Проверки Health Check

Код	Проверка	Содержание	Порог, мс
INTEGRITY	Целостность установки и данных	SHA-256 файлов активного слота, ключи appsettings.json, число записей не меньше, чем до обновления	–
DB	Локальная БД SQLite	PRAGMA integrity_check, версия схемы и обязательные колонки, пробная запись с откатом транзакции	200
API	Центральный API	GET https://api.ipify.org?format=json, таймаут 5 с на попытку, 3 попытки, IP маскируется	1500
READER	Считыватель карт	чтение тестовой карты (COM-порт или эмулятор), проверка формата UID	500

Источник: составлено автором

Ядро цепочки – класс HealthCheckRunner (листинг 1). Любое исключение внутри проверки превращается в результат «НЕ ПРОЙДЕНА», поэтому один сбой не обрывает цепочку, а акт готовности формируется всегда.

Листинг 1 – Выполнение проверки с замером задержки (HealthCheckRunner.RunAsync)

```
var sw = Stopwatch.StartNew();
try
{
   details = await check.RunAsync(context, ct);
   status = HealthStatus.Pass;
}
catch (OperationCanceledException) when (ct.IsCancellationRequested)
{
   throw; // прервано пользователем (Ctrl+C)
}
catch (Exception ex)
{
   status = HealthStatus.Fail;
   details = Describe(ex);          // понятный текст для журнала и акта
}
sw.Stop();
var latency = Math.Round(sw.Elapsed.TotalMilliseconds, 1);
var warnMs = check.WarnLatencyMs(context.Options.HealthCheck);
if (status == HealthStatus.Pass && warnMs > 0 && latency > warnMs)
   status = HealthStatus.Warn;      // медленно, но работает
```

3.3 Обработка исключений

Отдельное внимание уделено сетевым таймаутам и ошибкам файлового ввода-вывода (таблица 6). Обращение к API выполняется с таймаутом на каждую попытку через CancellationTokenSource.CancelAfter и повторами с экспоненциальной паузой 0,5, 1 и 2 с (листинг 2) [5].

Таблица 6 – Обрабатываемые исключительные ситуации

Ситуация	Исключение	Реакция утилиты
API не ответил за отведённое время	OperationCanceledException	повтор с паузой, после 3 попыток проверка API «НЕ ПРОЙДЕНА»
Нет сети, ошибка DNS, HTTP 5xx	HttpRequestException	повтор, затем «НЕ ПРОЙДЕНА» с текстом ошибки
Некорректный ответ API	JsonException, KeyNotFoundException, FormatException	«НЕ ПРОЙДЕНА: некорректный ответ API»
Файл пакета занят, сетевая папка недоступна	IOException	3 попытки копирования, затем прерывание без изменений
Нет прав на каталог	UnauthorizedAccessException	прерывание обновления, код 3
Пакет повреждён или подменён	InvalidDataException	пакет удаляется, обновление прервано, код 3
Ошибка БД или миграции	SqliteException	транзакция откатывается, данные восстанавливаются из копии
Считыватель не отвечает или порт занят	TimeoutException, UnauthorizedAccessException	проверка READER «НЕ ПРОЙДЕНА»
Ошибка в updater.json или аргументах	FileNotFoundException, InvalidDataException, ArgumentException	сообщение администратору, код 4

Источник: составлено автором

Листинг 2 – Таймаут и повторы при обращении к API (фрагмент ApiCheck.RunAsync)

```
for (var attempt = 1; attempt <= attempts; attempt++)
{
   using var cts = CancellationTokenSource.CreateLinkedTokenSource(ct);
   cts.CancelAfter(TimeSpan.FromSeconds(o.ApiTimeoutSeconds)); // таймаут попытки
   try
   {
      using var response = await http.GetAsync(o.ApiUrl, cts.Token);
      ...
      return $"GET {o.ApiUrl} → 200 OK, {sw.ElapsedMilliseconds} мс, IP {Mask(ip)}";
   }
   catch (OperationCanceledException) when (!ct.IsCancellationRequested)
   { lastError = $"таймаут {o.ApiTimeoutSeconds} с"; }
   catch (HttpRequestException ex)
   { lastError = $"сетевая ошибка: {ex.Message}"; }
   catch (Exception ex) when (ex is JsonException or KeyNotFoundException or
FormatException)
   { lastError = $"некорректный ответ API: {ex.Message}"; }

   if (attempt < attempts)                    // пауза 0,5 → 1 → 2 с
      await Task.Delay(TimeSpan.FromMilliseconds(500 * Math.Pow(2, attempt - 1)), ct);
}
throw new HttpRequestException($"API недоступен после {attempts} попыток
({lastError})");
```

3.4 Акт готовности АРМ

По итогам каждой операции формируется файл reports/ACT_<АРМ>_<дата_время>.md: параметры АРМ, версии до и после, активный слот, путь к резервной копии, таблица проверок со статусом и задержкой, статистика задержек (минимальная, средняя, максимальная), ход выполнения и заключение «ГОТОВ» или «НЕ ГОТОВ» с местами для подписей администратора и оператора. Коды завершения утилиты (0 – готов, 1 – проверки не пройдены, 2 – выполнен откат, 3 – обновление прервано, 4 – ошибка настроек) позволяют встроить её в планировщик заданий Windows.

4 ЭКСПЛУАТАЦИОННАЯ ДОКУМЕНТАЦИЯ

4.1 Состав документации

Эксплуатационная документация ведётся по принципу Docs-as-Code: файлы Markdown хранятся в том же репозитории, что и код, изменяются теми же коммитами и проходят ту же проверку. Структура документов следует ГОСТ 19 (ЕСПД) [6, 7, 8], журнал изменений – стандарту Keep a Changelog [9] и правилам семантического версионирования [10] (таблица 7).

Таблица 7 – Эксплуатационные документы

Документ	Основа	Содержание
OPERATOR_MANUAL.md – руководство оператора	ГОСТ 19.505-79	аннотация, назначение программы, условия выполнения, выполнение, сообщения оператору, действия при сбоях
UPDATE_GUIDE.md – инструкция по обновлению	ГОСТ 19.503-79	общие сведения, схема доставки, настройка, регламент, резервное копирование, план отката, проверка готовности
CHANGELOG.md – журнал изменений	Keep a Changelog 1.1.0, SemVer 2.0.0	версии 2.4.0–2.6.0 и Unreleased; разделы Added, Changed, Fixed, Breaking Changes; отозванные релизы помечены [YANKED]

Источник: составлено автором

В руководстве оператора нет команд утилиты – оператору достаточно знать сообщения программы и порядок действий; техническая часть вынесена в инструкцию администратора.

4.2 Примеры оформления документов

Вид документов в репозитории GitHub приведён на рисунках 6–11. Схема в UPDATE_GUIDE.md записана на языке Mermaid и отображается GitHub как диаграмма.

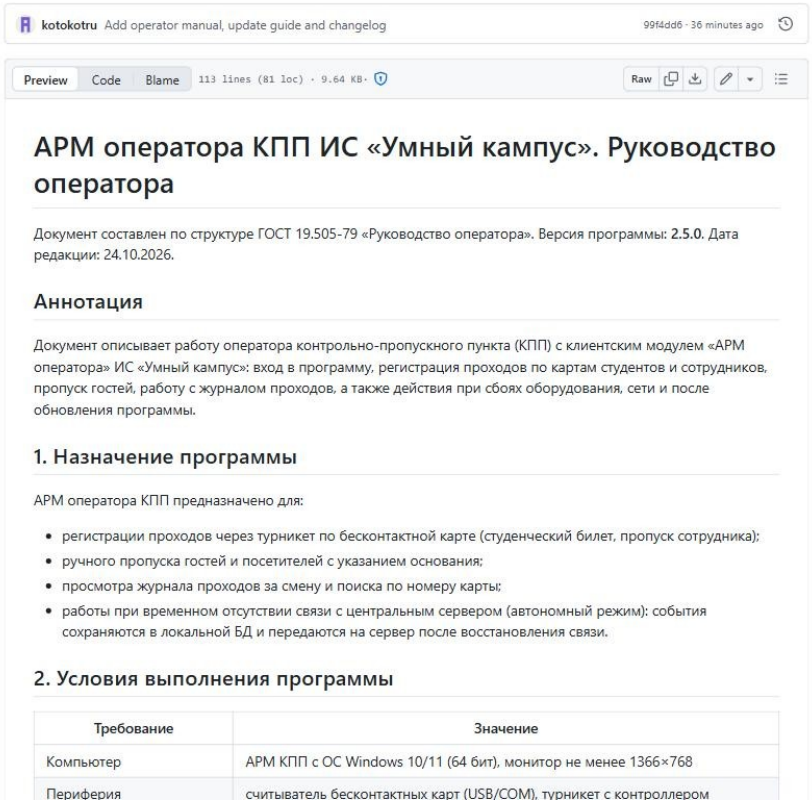
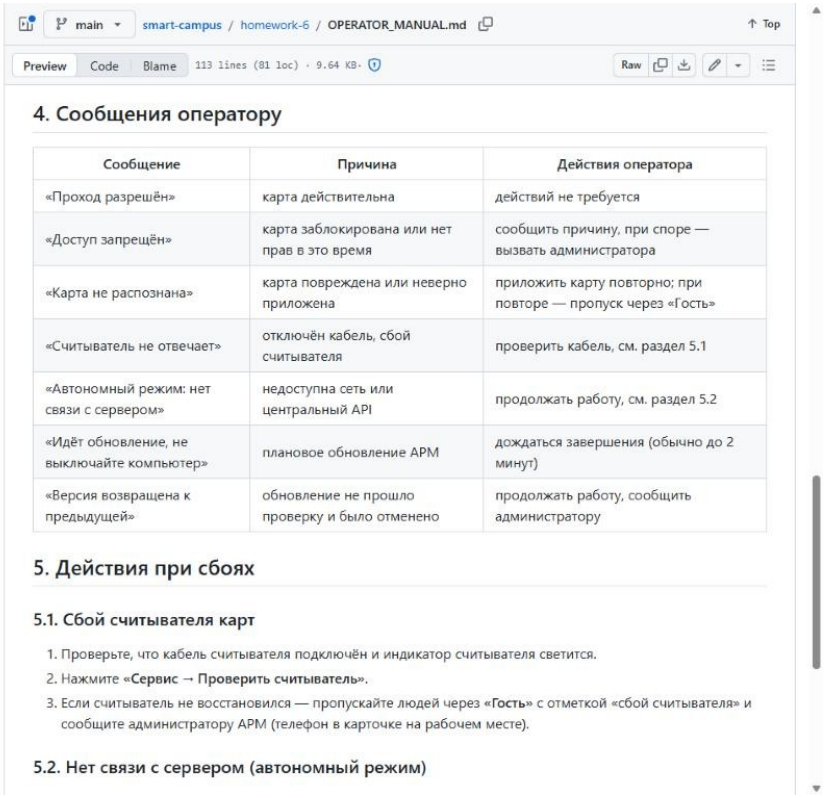


Рисунок 6 – Руководство оператора: аннотация и назначение программы



main smart-campus / homework-6 / OPERATOR_MANUAL.md ↑ Top

Preview Code Blame 113 lines (81 loc) · 9.64 KB Raw

4. Сообщения оператору

Сообщение	Причина	Действия оператора
«Проход разрешён»	карта действительна	действий не требуется
«Доступ запрещён»	карта заблокирована или нет прав в это время	сообщить причину, при споре — вызвать администратора
«Карта не распознана»	карта повреждена или неверно приложена	приложить карту повторно; при повторе — пропуск через «Гость»
«Считыватель не отвечает»	отключён кабель, сбой считывателя	проверить кабель, см. раздел 5.1
«Автономный режим: нет связи с сервером»	недоступна сеть или центральный API	продолжать работу, см. раздел 5.2
«Идёт обновление, не выключайте компьютер»	плановое обновление АРМ	дождаться завершения (обычно до 2 минут)
«Версия возвращена к предыдущей»	обновление не прошло проверку и было отменено	продолжать работу, сообщить администратору

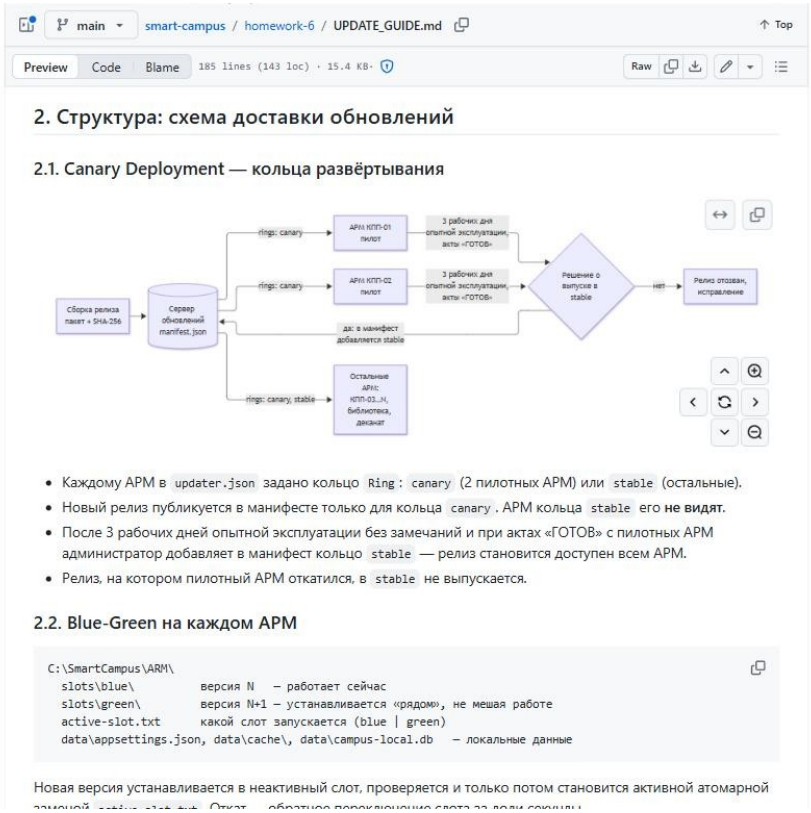
5. Действия при сбоях

5.1. Сбой считывателя карт

1. Проверьте, что кабель считывателя подключён и индикатор считывателя светится.
2. Нажмите **«Сервис → Проверить считыватель»**.
3. Если считыватель не восстановился — пропускайте людей через **«Гость»** с отметкой «сбой считывателя» и сообщите администратору АРМ (телефон в карточке на рабочем месте).

5.2. Нет связи с сервером (автономный режим)

Рисунок 7 – Руководство оператора: сообщения оператору и действия при сбоях



main smart-campus / homework-6 / UPDATE_GUIDE.md ↑ Top

Preview Code Blame 185 lines (143 loc) · 15.4 KB Raw

2. Структура: схема доставки обновлений

2.1. Canary Deployment — кольца развёртывания

- Каждому АРМ в `updater.json` задано кольцо `Ring`: `canary` (2 пилотных АРМ) или `stable` (остальные).
- Новый релиз публикуется в манифесте только для кольца `canary`. АРМ кольца `stable` его **не видят**.
- После 3 рабочих дней опытной эксплуатации без замечаний и при актах «ГОТОВ» с пилотных АРМ администратор добавляет в манифест кольцо `stable` — релиз становится доступен всем АРМ.
- Релиз, на котором пилотный АРМ откатился, в `stable` не выпускается.

2.2. Blue-Green на каждом АРМ

```
C:\SmartCampus\ARM\
  slots\blue\          версия N   — работает сейчас
  slots\green\         версия N+1 — устанавливается «рядом», не мешая работе
  active-slot.txt      какой слот запускается (blue | green)
  data\appsettings.json, data\cache\, data\campus-local.db   — локальные данные
```

Новая версия устанавливается в неактивный слот, проверяется и только потом становится активной атомарной

Рисунок 8 – Инструкция по обновлению: схема доставки и устройство Blue-Green

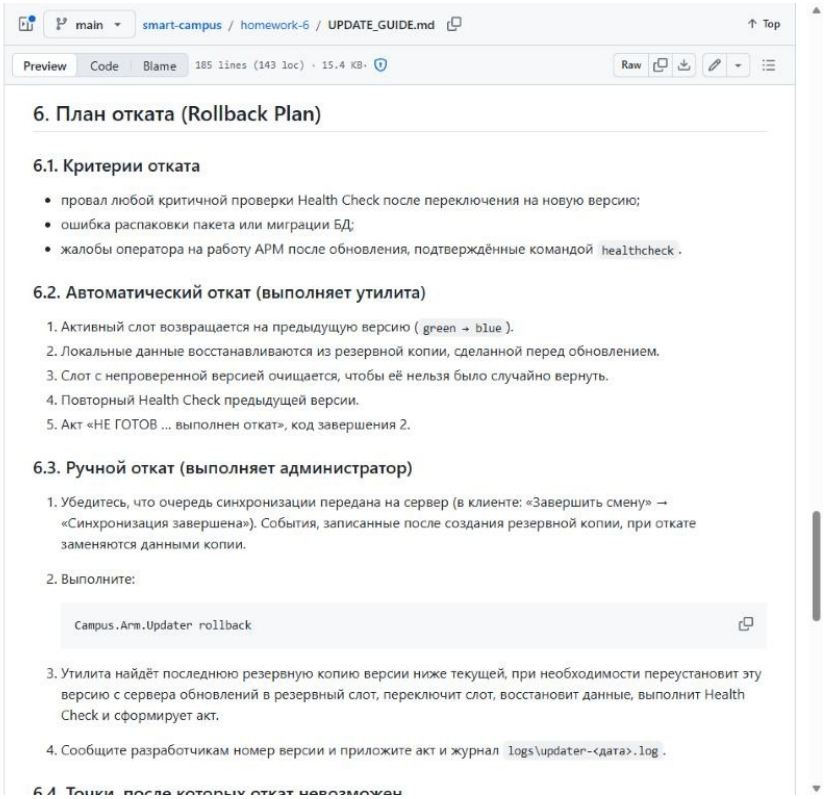


Рисунок 9 – Инструкция по обновлению: план отката

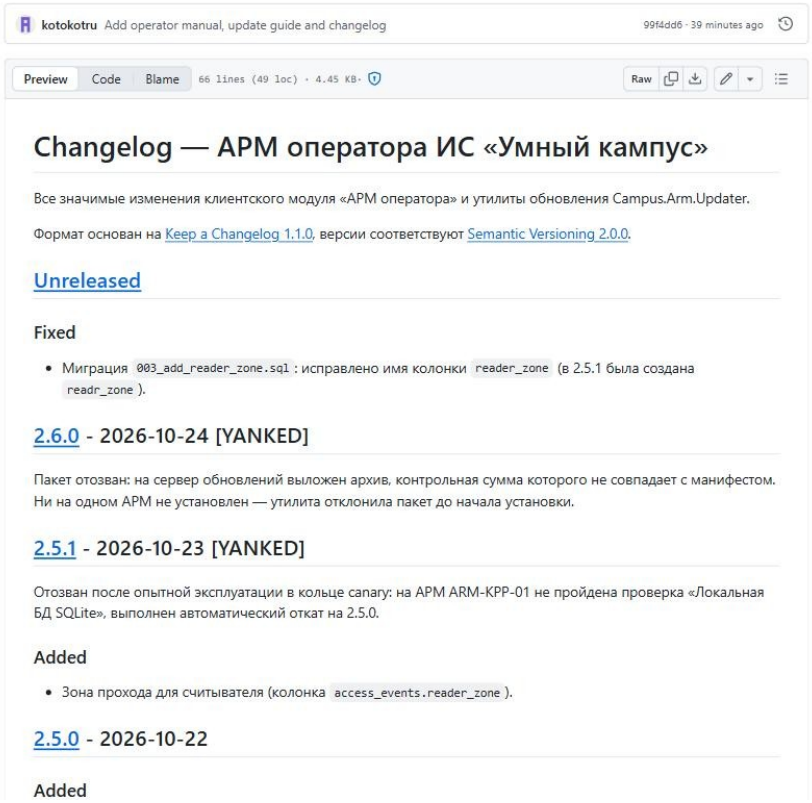


Рисунок 10 – Журнал изменений: Unreleased и отозванные релизы

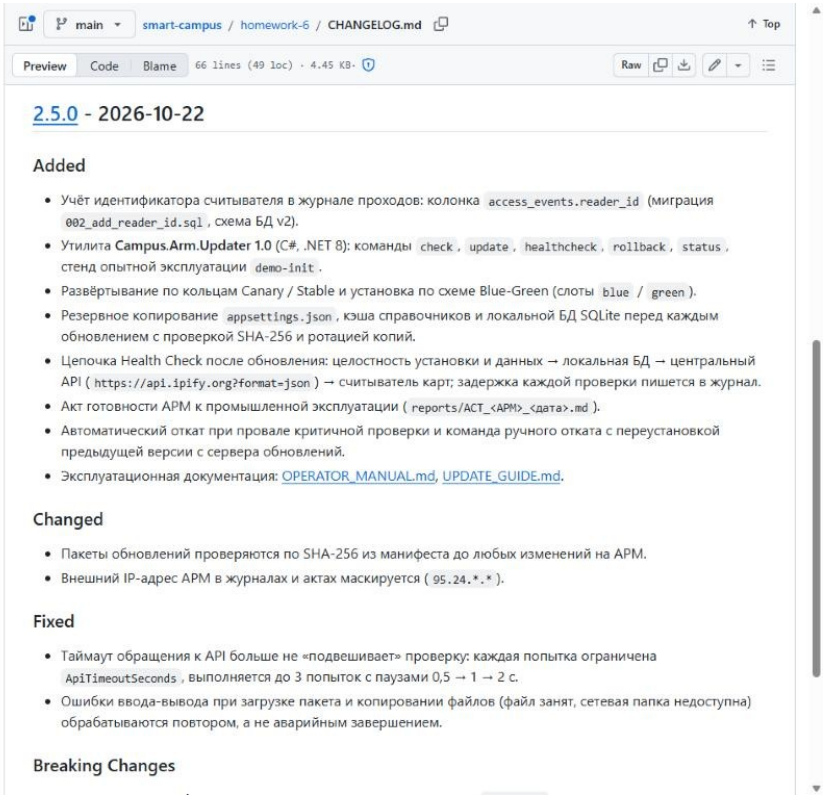


Рисунок 11 – Журнал изменений: запись о релизе 2.5.0

Запись [2.5.1] [YANKED] в журнале изменений показывает связь документации с результатами эксплуатации: релиз отозван по акту автоматического отката с пилотного АРМ, а исправление миграции попало в раздел Unreleased.

5 SANITY-ТЕСТ И ИСПЫТАНИЯ НА СТЕНДЕ

5.1 Стенд опытной эксплуатации

Команда demo-init создаёт стенд: АРМ ARM-KPP-01 (кольцо canary) с версией 2.4.0 в слоте blue, локальной БД (3 оператора, 1200 событий прохода, 25 записей в очереди синхронизации) и сервер обновлений с релизами 2.5.0 (исправный), 2.5.1 (ошибка в миграции – опечатка в имени колонки) и 2.6.0 (повреждённый пакет, контрольная сумма не совпадает с манифестом). Все запуски выполнены на рабочем компьютере с реальным обращением к api.ipify.org (рисунок 12).

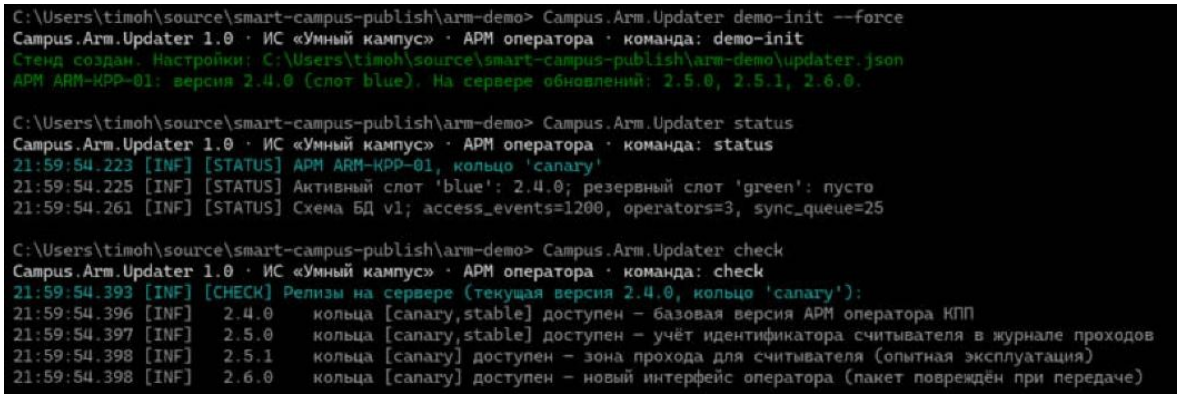


Рисунок 12 – Создание стенда, состояние АРМ и доступные релизы

5.2 Штатное обновление

При обновлении 2.4.0 → 2.5.0 пакет загружен и проверен по SHA-256, создана резервная копия, версия установлена в слот green, применена миграция 002_add_reader_id.sql, слот переключён. Все четыре проверки пройдены, число записей до и после обновления совпадает, задержка API – 551 мс. Акт: «АРМ ГОТОВ к промышленной эксплуатации (версия 2.5.0)», код 0 (рисунок 13).

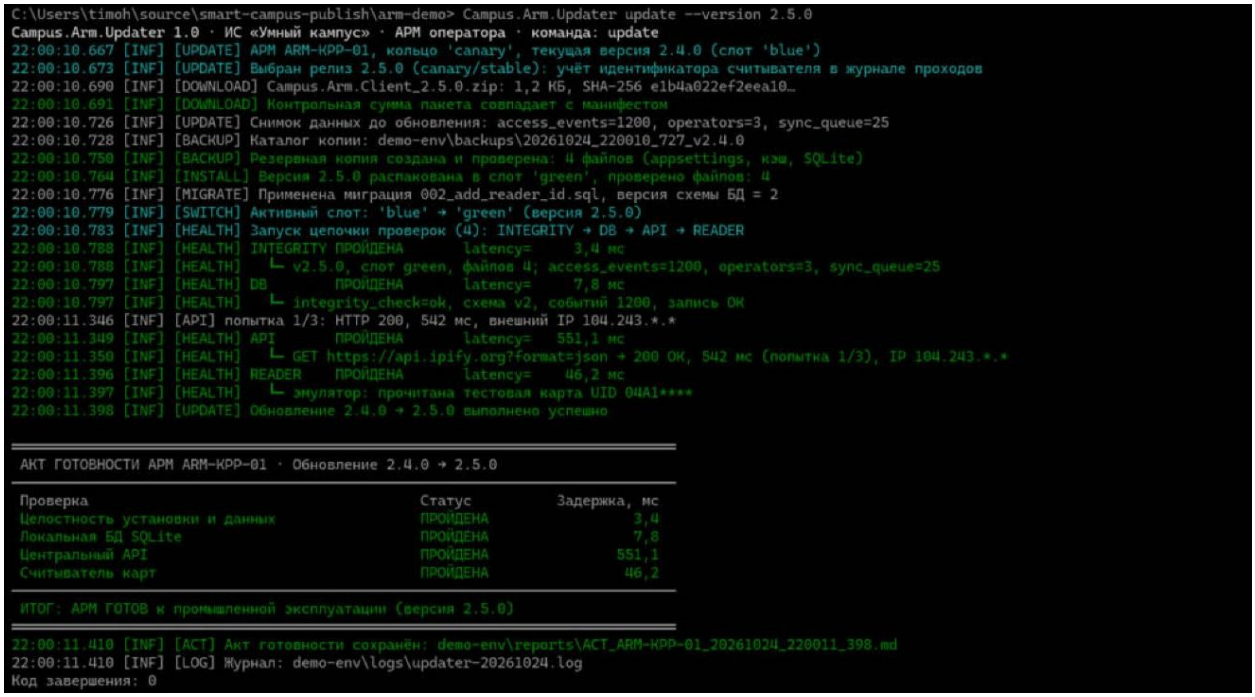


Рисунок 13 – Штатное обновление 2.4.0 → 2.5.0 и акт «ГОТОВ»

5.3 Сбой обновления и автоматический откат

При обновлении 2.5.0 → 2.5.1 после переключения на слот blue не пройдена проверка DB: в базе нет колонки access_events.reader_zone, которую ожидает версия 2.5.1 (миграция создала readr_zone). Утилита вернула слот green, восстановила данные из копии, очистила слот с непроверенной версией и повторно проверила версию 2.5.0 – все проверки пройдены. Акт: «АРМ НЕ ГОТОВ к версии 2.5.1 – выполнен ОТКАТ на 2.5.0», код 2 (рисунок 14).

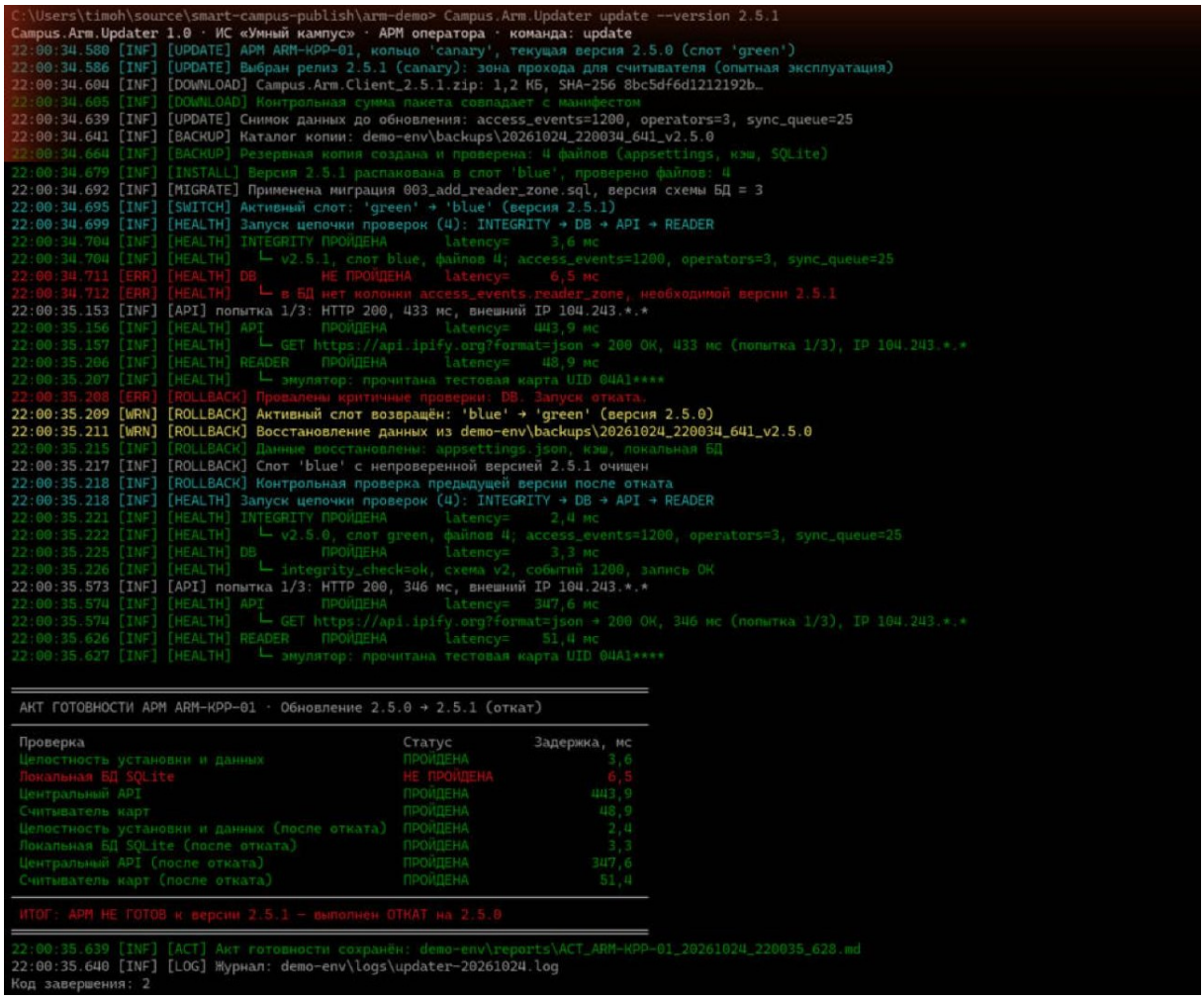


Рисунок 14 – Провал проверки БД после обновления до 2.5.1 и автоматический откат

5.4 Повреждённый пакет и сетевой таймаут

Для пакета 2.6.0 контрольная сумма не совпала с манифестом: обновление прервано до установки, на АРМ ничего не изменилось, работа продолжается на версии 2.5.0, код 3 (рисунок 15).

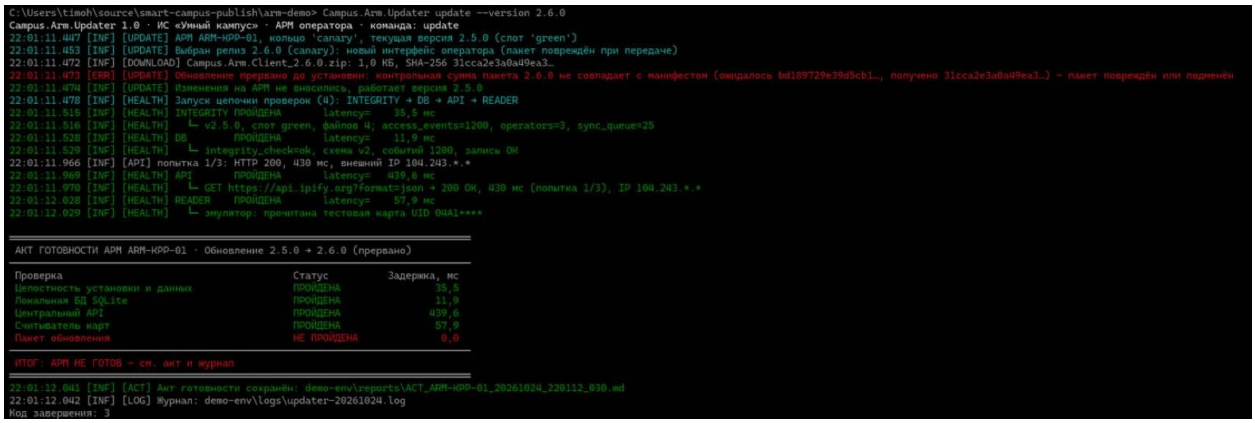


Рисунок 15 – Отклонение повреждённого пакета 2.6.0 до установки

Для проверки обработки таймаута адрес API подменён на недоступный с таймаутом 2 с. Выполнены три попытки с паузами 500 и 1000 мс, проверка API не пройдена, остальные проверки выполнены, акт «НЕ ГОТОВ», код 1 – утилита не зависла и не завершилась аварийно (рисунок 16).

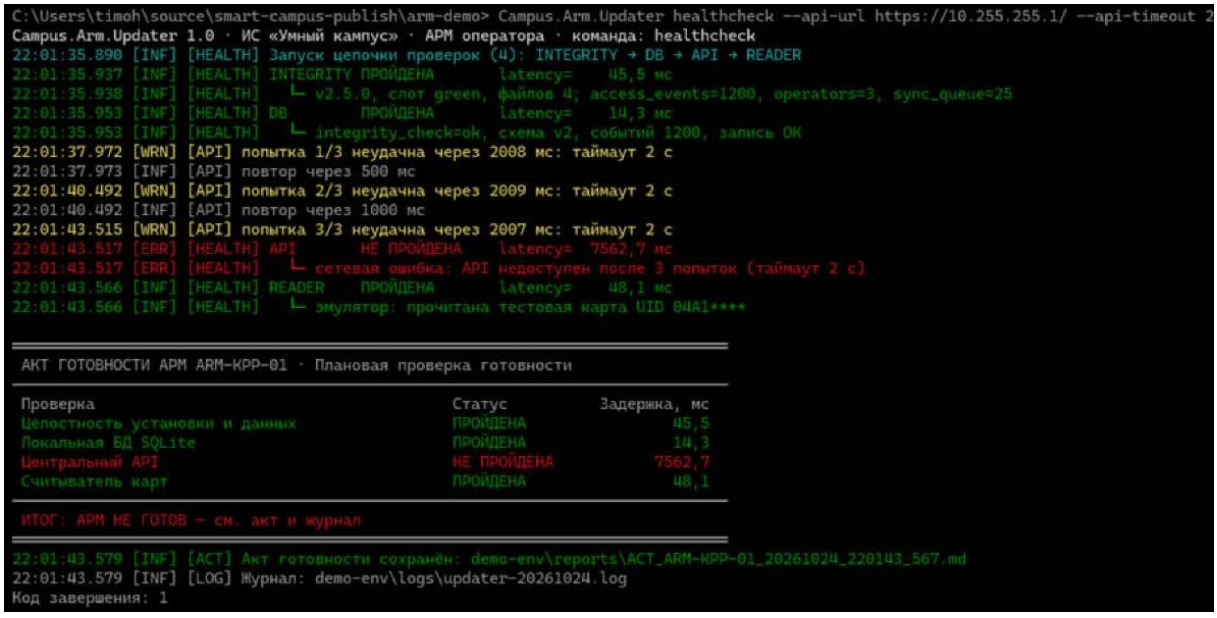


Рисунок 16 – Обработка сетевого таймаута

5.5 Ручной откат

Команда rollback нашла резервную копию версии 2.4.0, заново установила эту версию с сервера в слот blue (слот был очищен после неудачного обновления до 2.5.1), переключила слот и восстановила данные. Задержка API 1682 мс превысила порог 1500 мс – статус «ПРЕДУПРЕЖДЕНИЕ», который не блокирует готовность. Команда status подтверждает версию 2.4.0 и схему БД v1 (рисунок 17).

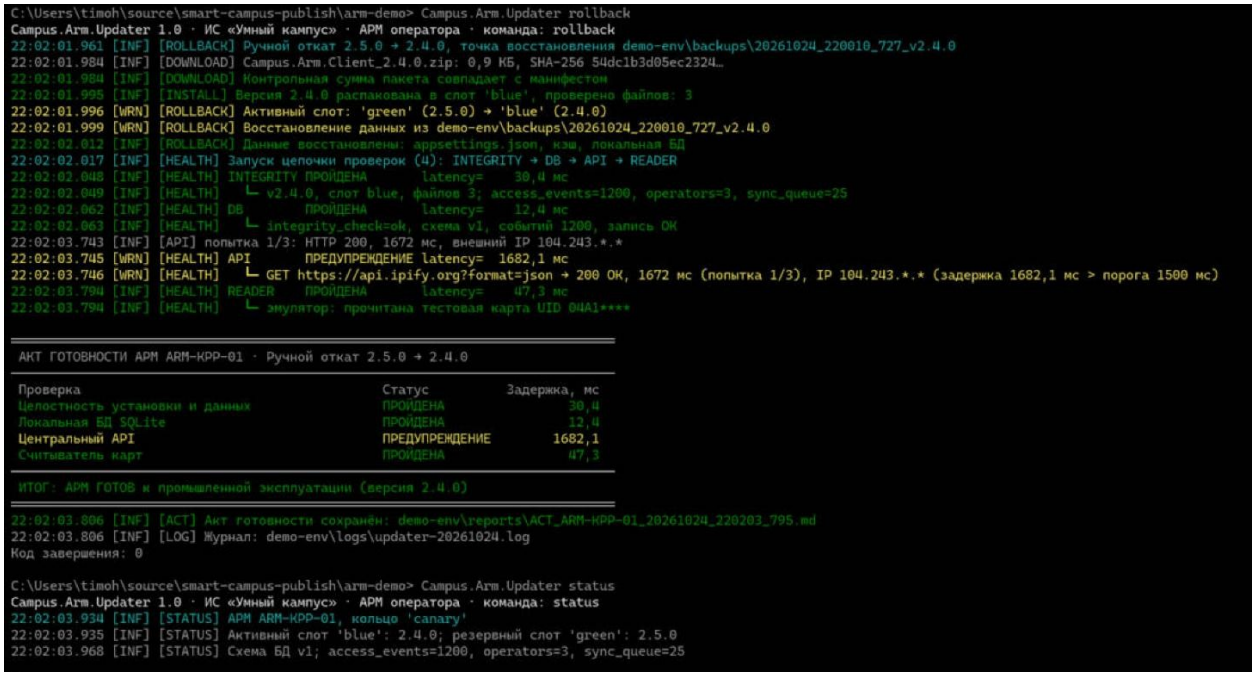


Рисунок 17 – Ручной откат на версию 2.4.0 и проверка состояния АРМ

5.6 Акт готовности

Фрагмент акта, сформированного при автоматическом откате, приведён в листинге 3; полные примеры актов находятся в папке docs/examples репозитория.

Листинг 3 – Акт готовности АРМ при откате (фрагмент)

```
# АКТ готовности АРМ к промышленной эксплуатации

ИС «Умный кампус» · клиентский модуль «АРМ оператора»

| Параметр | Значение |
|---|---|
| АРМ | ARM-KPP-01 (DESKTOP-9QU5525) |
| Операция | Обновление 2.5.0 → 2.5.1 (откат) |
| Версия до / после | 2.5.0 / 2.5.1 |
| Активный слот (Blue-Green) | green |
| Кольцо развёртывания | canary |
| Резервная копия | demo-env\backups\20261024_220034_641_v2.5.0 |
| Начало / окончание | 24.10.2026 22:00:34 / 24.10.2026 22:00:35 |

## Результаты проверок (Health Check)

| № | Проверка | Критичная | Статус | Задержка, мс | Результат |
|---|---|---|---|---|---|
| 1 | Целостность установки и данных | да | ПРОЙДЕНА | 3,6 | v2.5.1, слот blue, файлов 4; access_events=1200, operators=3, sync_queue=25 |
| 2 | Локальная БД SQLite | да | НЕ ПРОЙДЕНА | 6,5 | в БД нет колонки access_events.reader_zone, необходимой версии 2.5.1 |
| 3 | Центральный API | да | ПРОЙДЕНА | 443,9 | GET https://api.ipify.org?format=json → 200 OK, 433 мс (попытка 1/3), IP 104.243.*.* |
| 4 | Считыватель карт | да | ПРОЙДЕНА | 48,9 | эмулятор: прочитана тестовая карта UID 04A1**** |

Задержка проверок: мин. 2,4 мс, средн. 113,5 мс, макс. 443,9 мс, суммарно 907,6 мс.

## Ход выполнения
```

5.7 Автоматизированный sanity-тест

Класс SanityTests (xUnit) имитирует установку обновления на стенде во временном каталоге и проверяет целостность данных и акт готовности. Внешний API заменён заглушкой, поэтому тесты не зависят от интернета (таблица 8).

Таблица 8 – Сценарии sanity-теста

Тест	Что проверяется
Update_GoodRelease_SwitchesSlot_KeepsData_AndActSaysReady	слот переключён, схема v2, 1200 событий сохранены, акт «ГОТОВ», копия создана
Update_BrokenMigration_RollsBack_RestoresDatabase	откат на 2.5.0, схема восстановлена из копии, ошибочной колонки нет
ManualRollback_AfterFailedUpdate_ReturnsToPreviousGoodVersion	ручной откат после сбоя возвращает 2.4.0, а не непроверенную 2.5.1
Update_CorruptedPackage_IsRejected_BeforeAnyChange	код 3, слот и данные не изменились
HealthCheck_ApiTimeout_IsHandled_AndArmIsNotReady	таймаут API обработан, акт «НЕ ГОТОВ», код 1
Update_ApiDownAfterSwitch_TriggersRollback	недоступность API после переключения вызывает откат
HealthCheck_MissingReaderCard_FailsReaderOnly	сбой считывателя не влияет на остальные проверки
ExternalIp_IsMasked	внешний IP в журнале и акте маскируется

Источник: составлено автором

Результат запуска тестов в Visual Studio – 8 из 8 пройдено (рисунок 18). Тот же набор тестов автоматически выполняется в GitHub Actions на каждый push (рисунки 19 и 20).

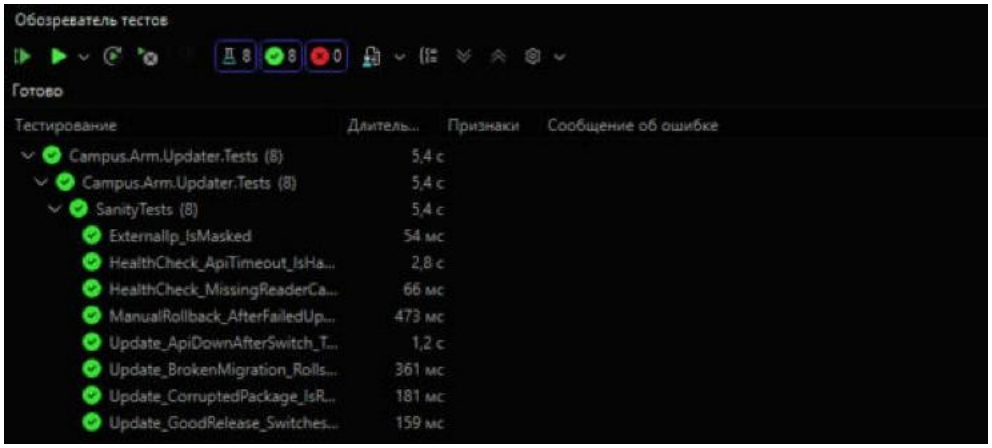


Рисунок 18 – Обозреватель тестов Visual Studio: 8 тестов пройдено

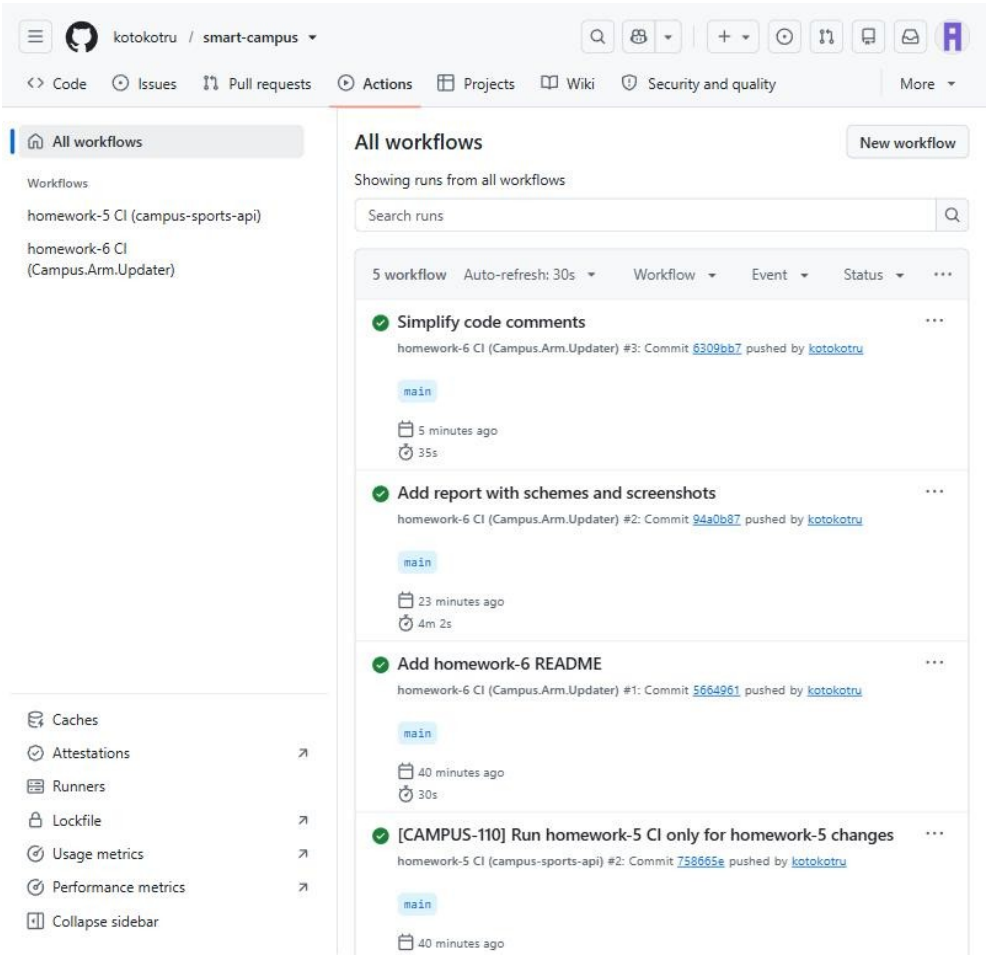


Рисунок 19 – GitHub Actions: запуски workflow homework-6 CI

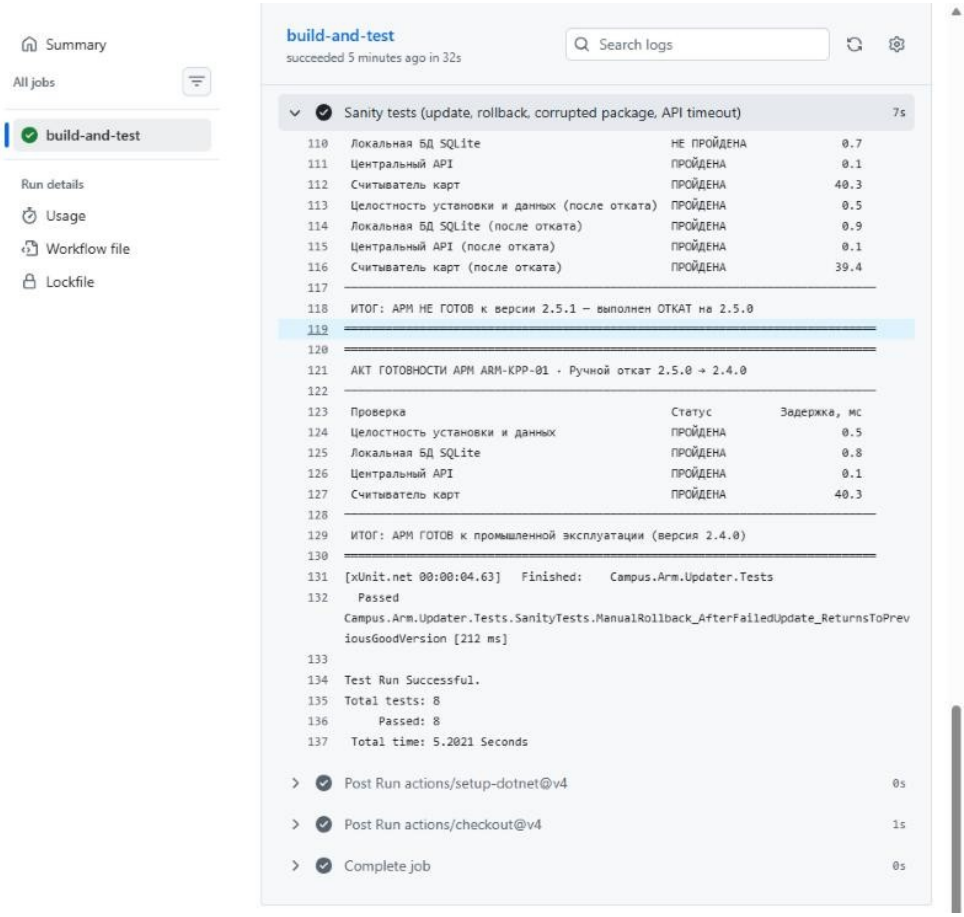


Рисунок 20 – Журнал задания build-and-test: 8 тестов пройдено

ЗАКЛЮЧЕНИЕ

В ходе практической работы для клиентского модуля «АРМ оператора» ИС «Умный кампус» организовано сопровождение на этапе эксплуатации в соответствии с принципами CALS-технологий.

Разработан регламент управления обновлениями: доставка по кольцам Canary с опытной эксплуатацией на пилотных АРМ и установка по схеме Blue-Green без остановки работы оператора, обязательное резервное копирование настроек, кэша и локальной БД SQLite с контролем SHA-256, план автоматического и ручного отката с определёнными критериями и точками невозврата. Схемы регламента выполнены в draw.io.

По варианту разработана утилита «Тестер Health Check после обновления» на C# .NET 8: цепочка проверок целостности, БД, API и считывателя с журналированием задержек и формированием акта готовности. Утилита корректно обрабатывает сетевые таймауты, ошибки соединения и файлового ввода-вывода, повреждённые пакеты и ошибки миграций. Испытания на стенде и 8 автоматизированных sanity-тестов подтвердили штатное обновление без потери данных, автоматический откат при сбое и отклонение повреждённого пакета.

Выводы о применении CALS на этапе эксплуатации:

– единое информационное пространство: код, регламент, руководства, журнал изменений и акты хранятся в одном репозитории и изменяются согласованно;

– электронная история жизненного цикла: каждое обновление оставляет журнал, акт готовности с задержками проверок и запись в журнале изменений;

– управление конфигурацией: манифест с контрольными суммами, версии схемы БД и семантическое версионирование однозначно определяют состав каждой версии;

– обратная связь с разработкой: акт отката с пилотного АРМ привёл к отзыву релиза 2.5.1 и исправлению в следующей версии.

В дальнейшем утилиту можно дополнить отправкой актов готовности на центральный сервер и сводной панелью состояния всех АРМ кампуса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ Р 50.1.031-2001. Информационные технологии поддержки жизненного цикла продукции. Терминологический словарь. Часть 1. Стадии жизненного цикла продукции : рекомендации по стандартизации : дата введения 2002-07-01. – Москва : Госстандарт России, 2001. – Текст : непосредственный.

2. ГОСТ Р 57193-2016. Системная и программная инженерия. Процессы жизненного цикла систем : национальный стандарт Российской Федерации : дата введения 2017-11-01. – Москва : Стандартинформ, 2016. – Текст : непосредственный.

3. Fowler, M. Canary Release / M. Fowler // martinfowler.com : сайт. – URL: https://martinfowler.com/bliki/CanaryRelease.html (дата обращения: 24.10.2026). – Текст : электронный.

4. Fowler, M. Blue Green Deployment / M. Fowler // martinfowler.com : сайт. – URL: https://martinfowler.com/bliki/BlueGreenDeployment.html (дата обращения: 24.10.2026). – Текст : электронный.

5. HttpClient.Timeout. Свойство // Microsoft Learn : сайт. – URL: https://learn.microsoft.com/ru-ru/dotnet/api/system.net.http.httpclient.timeout (дата обращения: 24.10.2026). – Текст : электронный.

6. ГОСТ 19.505-79. Единая система программной документации. Руководство оператора. Требования к содержанию и оформлению : межгосударственный стандарт : дата введения 1980-01-01. – Москва : Стандартинформ, 2010. – Текст : непосредственный.

7. ГОСТ 19.503-79. Единая система программной документации. Руководство системного программиста. Требования к содержанию и оформлению : межгосударственный стандарт : дата введения 1980-01-01. – Москва : Стандартинформ, 2010. – Текст : непосредственный.

8. ГОСТ 19.106-78. Единая система программной документации. Требования к программным документам, выполненным печатным способом :

межгосударственный стандарт : дата введения 1980-01-01. – Москва : Стандартинформ, 2010. – Текст : непосредственный.

9. Keep a Changelog 1.1.0 // keepachangelog.com : сайт. – URL: https://keepachangelog.com/ru/1.1.0/ (дата обращения: 24.10.2026). – Текст : электронный.

10. Семантическое версионирование 2.0.0 // semver.org : сайт. – URL: https://semver.org/lang/ru/ (дата обращения: 24.10.2026). – Текст : электронный.

11. Online Backup. Microsoft.Data.Sqlite // Microsoft Learn : сайт. – URL: https://learn.microsoft.com/ru-ru/dotnet/standard/data/sqlite/backup (дата обращения: 24.10.2026). – Текст : электронный.

12. ipify API // ipify.org : сайт. – URL: https://www.ipify.org/ (дата обращения: 24.10.2026). – Текст : электронный.

ПРИЛОЖЕНИЕ А

Исходный код утилиты Campus.Arm.Updater

В приложении приведён исходный код консольной утилиты Campus.Arm.Updater (C#, .NET 8, решение Microsoft Visual Studio Campus.Arm.Updater.sln). Генератор демонстрационного стенда Demo/DemoEnvironment.cs и файлы проектов находятся в репозитории.

Листинг А.1 – Файл Campus.Arm.Updater/Program.cs

```
// Campus.Arm.Updater — утилита обновления и проверки готовности АРМ оператора
// ИС «Умный кампус». Вариант: «Тестер Health Check после обновления» — цепочка проверок
// (БД, API, считыватель), журнал задержек (Latency), акт готовности АРМ.
//
// Команды:
//   demo-init   [--force]              создать стенд опытной эксплуатации (АРМ 2.4.0 + сервер обновлений)
//   status                             версия, активный слот, состояние локальной БД
//   check                              релизы на сервере обновлений и их доступность для кольца АРМ
//   update      [--version X.Y.Z]      обновление: backup → установка → Health Check → акт (откат при сбое)
//   healthcheck                        только цепочка проверок и акт готовности
//   rollback                           ручной откат на версию из резервного слота
// Общие параметры:
//   --config <путь>        файл настроек (по умолчанию ./updater.json)
//   --api-url <url>        переопределить адрес API (например, для имитации недоступности сети)
//   --api-timeout <сек>    таймаут одной попытки запроса к API
//   --reader <Emulator|Serial>

using System.Text;
using Campus.Arm.Updater.Config;
using Campus.Arm.Updater.Demo;
using Campus.Arm.Updater.Infrastructure;
using Campus.Arm.Updater.Update;

Console.OutputEncoding = Encoding.UTF8;

var command = args.Length > 0 ? args[0].ToLowerInvariant() : "help";
var opts = ParseOptions(args.Skip(1).ToArray());

using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) =>
{
    e.Cancel = true;            // корректно завершаем текущий шаг вместо «убийства» процесса
    cts.Cancel();
};

PrintBanner(command);

try
{
    if (command is "help" or "-h" or "--help")
    {
        PrintHelp();
        return ExitCodes.Ok;
    }
```

```
    if (command == "demo-init")
    {
        var config = DemoEnvironment.Create(Directory.GetCurrentDirectory(), force: opts.ContainsKey("force"));
        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine($"Стенд создан. Настройки: {config}");
        Console.WriteLine("APM ARM-KPP-01: версия 2.4.0 (слот blue). На сервере обновлений: 2.5.0, 2.5.1,
2.6.0.");
        Console.ResetColor();
        return ExitCodes.Ok;
    }

    var options = UpdaterOptions.Load(opts.GetValueOrDefault("config", "updater.json"));
    if (opts.TryGetValue("api-url", out var apiUrl)) options.HealthCheck.ApiUrl = apiUrl;
    if (opts.TryGetValue("api-timeout", out var timeout)) options.HealthCheck.ApiTimeoutSeconds =
int.Parse(timeout);
    if (opts.TryGetValue("reader", out var reader)) options.HealthCheck.Reader.Mode = reader;

    var log = new AppLogger(options.LogsRoot);
    var orchestrator = new UpdateOrchestrator(options, log);

    return command switch
    {
        "status" => orchestrator.Status(),
        "check" => orchestrator.Check(),
        "update" => await orchestrator.UpdateAsync(opts.GetValueOrDefault("version"), cts.Token),
        "healthcheck" => await orchestrator.HealthCheckAsync(cts.Token),
        "rollback" => await orchestrator.RollbackAsync(cts.Token),
        _ => Unknown(command)
    };
}
catch (OperationCanceledException)
{
    Console.Error.WriteLine("Операция прервана пользователем (Ctrl+C).");
    return ExitCodes.Aborted;
}
catch (Exception ex) when (ex is FileNotFoundException or InvalidDataException or FormatException or
ArgumentException)
{
    Console.ForegroundColor = ConsoleColor.Red;
    Console.Error.WriteLine($"Ошибка настройки/данных: {ex.Message}");
    Console.ResetColor();
    return ExitCodes.ConfigError;
}
catch (Exception ex) when (ex is IOException or UnauthorizedAccessException)
{
    Console.ForegroundColor = ConsoleColor.Red;
    Console.Error.WriteLine($"Ошибка файловой системы: {ex.Message}");
    Console.ResetColor();
    return ExitCodes.Aborted;
}

static Dictionary<string, string> ParseOptions(string[] a)
{
    var result = new Dictionary<string, string>(StringComparer.OrdinalIgnoreCase);
    for (var i = 0; i < a.Length; i++)
    {
        if (!a[i].StartsWith("--")) continue;
        var key = a[i][2..];
        var hasValue = i + 1 < a.Length && !a[i + 1].StartsWith("--");
        result[key] = hasValue ? a[++i] : "true";
    }
    return result;
}
```

```
static int Unknown(string command)
{
   Console.Error.WriteLine($"Неизвестная команда: {command}");
   PrintHelp();
   return ExitCodes.ConfigError;
}

static void PrintBanner(string command)
{
   Console.ForegroundColor = ConsoleColor.White;
   Console.WriteLine($"Campus.Arm.Updater 1.0 · ИС «Умный кампус» · АРМ оператора · команда:
{command}");
   Console.ResetColor();
}

static void PrintHelp()
{
   Console.WriteLine("""
      Использование: Campus.Arm.Updater <команда> [параметры]

        demo-init [--force]        создать стенд (АРМ 2.4.0 + сервер обновлений с релизами 2.5.0, 2.5.1, 2.6.0)
        status                     текущая версия, слот Blue-Green, состояние локальной БД
        check                      релизы на сервере и их доступность для кольца АРМ (canary/stable)
        update [--version X.Y.Z]   обновить АРМ (резервная копия → установка → Health Check → акт / откат)
        healthcheck                выполнить цепочку проверок и сформировать акт готовности
        rollback                   вручную вернуть предыдущую версию

        --config <путь>            файл настроек (по умолчанию ./updater.json)
        --api-url <url>            адрес API для проверки (по умолчанию https://api.ipify.org?format=json)
        --api-timeout <сек>        таймаут одной попытки обращения к API
        --reader <Emulator|Serial>  режим считывателя карт

      Коды завершения: 0 — готов, 1 — не готов, 2 — выполнен откат, 3 — прервано, 4 — ошибка настроек
      """);
}
```

Листинг А.2 – Файл Campus.Arm.Updater/Config/UpdaterOptions.cs

```
using System.Text.Json;
using Campus.Arm.Updater.Infrastructure;

namespace Campus.Arm.Updater.Config;

/// <summary>
/// Настройки утилиты (файл updater.json рядом с утилитой или путь из --config).
/// Все относительные пути считаются от папки, в которой лежит файл настроек.
/// </summary>
public sealed class UpdaterOptions
{
   /// <summary>Идентификатор АРМ (попадает в журнал и акт готовности).</summary>
   public string ArmId { get; set; } = "ARM-KPP-01";

   /// <summary>
   /// Кольцо развёртывания (Canary Deployment): "canary" — пилотные АРМ получают релиз первыми,
   /// "stable" — остальные АРМ получают релиз после успешной опытной эксплуатации на canary.
   /// </summary>
   public string Ring { get; set; } = "canary";

   /// <summary>Каталог установки клиента АРМ (слоты blue/green, локальные данные).</summary>
   public string InstallRoot { get; set; } = "demo-env/arm";
```

```
    /// <summary>Источник обновлений — сетевая папка центрального сервера (\\server\updates) или локальный каталог.</summary>
    public string UpdateSource { get; set; } = "demo-env/update-server";

    public string BackupRoot { get; set; } = "demo-env/backups";
    public string ReportsRoot { get; set; } = "demo-env/reports";
    public string LogsRoot { get; set; } = "demo-env/logs";

    /// <summary>Сколько последних резервных копий хранить.</summary>
    public int BackupsToKeep { get; set; } = 5;

    public HealthCheckOptions HealthCheck { get; set; } = new();

    /// <summary>Загрузка настроек. Ошибки формата превращаются в понятное сообщение для администратора.</summary>
    public static UpdaterOptions Load(string path)
    {
        if (!File.Exists(path))
            throw new FileNotFoundException($"Файл настроек не найден: {path}. Выполните команду demo-init или укажите --config.", path);

        UpdaterOptions? options;
        try
        {
            options = JsonSerializer.Deserialize<UpdaterOptions>(File.ReadAllText(path), Json.Options);
        }
        catch (JsonException ex)
        {
            throw new InvalidDataException($"Ошибка в файле настроек {path}: {ex.Message}", ex);
        }

        options ??= new UpdaterOptions();
        options.ResolvePaths(Path.GetDirectoryName(Path.GetFullPath(path))!);
        return options;
    }

    /// <summary>Приводит относительные пути к абсолютным.</summary>
    public void ResolvePaths(string baseDir)
    {
        InstallRoot = Path.GetFullPath(InstallRoot, baseDir);
        UpdateSource = Path.GetFullPath(UpdateSource, baseDir);
        BackupRoot = Path.GetFullPath(BackupRoot, baseDir);
        ReportsRoot = Path.GetFullPath(ReportsRoot, baseDir);
        LogsRoot = Path.GetFullPath(LogsRoot, baseDir);
    }
}

/// <summary>Параметры цепочки проверок Health Check.</summary>
public sealed class HealthCheckOptions
{
    /// <summary>Внешний API, доступность которого проверяется (по варианту — api.ipify.org).</summary>
    public string ApiUrl { get; set; } = "https://api.ipify.org?format=json";

    /// <summary>Таймаут одной попытки запроса к API, секунд.</summary>
    public int ApiTimeoutSeconds { get; set; } = 5;

    /// <summary>Количество попыток (повтор с экспоненциальной паузой 0,5 → 1 → 2 с).</summary>
    public int ApiRetries { get; set; } = 3;

    /// <summary>Пороги задержки (Latency), после которых проверка получает статус «Предупреждение».</summary>
    public int DatabaseWarnMs { get; set; } = 200;
    public int ApiWarnMs { get; set; } = 1500;
```

```
    public int ReaderWarnMs { get; set; } = 500;

    public ReaderOptions Reader { get; set; } = new();
}

/// <summary>Параметры считывателя карт (турникет / пропускной пункт).</summary>
public sealed class ReaderOptions
{
    /// <summary>"Emulator" — эмулятор (стенд, опытная эксплуатация), "Serial" — реальный считыватель на COM-порту.</summary>
    public string Mode { get; set; } = "Emulator";
    public string PortName { get; set; } = "COM3";
    public int BaudRate { get; set; } = 9600;
    public int TimeoutMs { get; set; } = 2000;
}
```

Листинг А.3 – Файл Campus.Arm.Updater/Update/UpdateOrchestrator.cs

```
using System.Text.Json;
using Campus.Arm.Updater.Config;
using Campus.Arm.Updater.Health;
using Campus.Arm.Updater.Infrastructure;
using Campus.Arm.Updater.Reporting;

namespace Campus.Arm.Updater.Update;

/// <summary>
/// Регламент обновления АРМ (Patch Management), реализованный в коде:
/// <list type="number">
/// <item>получение манифеста с сервера обновлений и выбор релиза для кольца АРМ (Canary / Stable);</item>
/// <item>загрузка пакета и проверка SHA-256 — до любых изменений на АРМ;</item>
/// <item>резервное копирование appsettings.json, кэша и SQLite;</item>
/// <item>установка в неактивный слот (Blue-Green) и миграции БД;</item>
/// <item>переключение слота и цепочка Health Check (целостность → БД → API → считыватель);</item>
/// <item>при провале критичной проверки — автоматический откат: слот назад + восстановление данных;</item>
/// <item>формирование акта готовности.</item>
/// </list>
/// </summary>
public sealed class UpdateOrchestrator
{
    private readonly UpdaterOptions _options;
    private readonly AppLogger _log;
    private readonly ArmLayout _layout;
    private readonly Func<HttpMessageHandler>? _apiHandler;

    public UpdateOrchestrator(UpdaterOptions options, AppLogger log, Func<HttpMessageHandler>? apiHandler = null)
    {
        _options = options;
        _log = log;
        _layout = new ArmLayout(options.InstallRoot);
        _apiHandler = apiHandler;
    }

    /// <summary>Путь к последнему сформированному акту (для тестов и вывода в консоль).</summary>
    public string? LastActPath { get; private set; }

    // Команда update
    public async Task<int> UpdateAsync(string? targetVersion, CancellationToken ct = default)
    {
```

```
        var started = DateTime.Now;
        var notes = new List<string>();
        var activeSlot = _layout.ReadActiveSlot();
        var current = _layout.ReadVersion(activeSlot)
                ?? throw new InvalidDataException($"В активном слоте '{activeSlot}' нет установленной версии");

        _log.Step($"[UPDATE] АРМ {_options.ArmId}, кольцо '{_options.Ring}', текущая версия
{current.Version} (слот '{activeSlot}')");

        // 1. Манифест сервера обновлений
        ReleaseInfo? release;
        try
        {
            release = SelectRelease(LoadManifest(), current, targetVersion);
        }
        catch (Exception ex) when (ex is IOException or UnauthorizedAccessException or JsonException or
InvalidDataException)
        {
            _log.Error($"[UPDATE] Сервер обновлений недоступен или манифест повреждён: {ex.Message}");
            return ExitCodes.Aborted;
        }

        if (release is null) return ExitCodes.Ok;
        _log.Step($"[UPDATE] Выбран релиз {release.Version} ({string.Join("/", release.Rings)}):
{release.Notes}");
        notes.Add($"Релиз {release.Version} получен с сервера обновлений ({release.Notes})");

        // 2. Загрузка пакета и контроль SHA-256
        string package;
        try
        {
            package = Download(release);
        }
        catch (Exception ex) when (ex is IOException or UnauthorizedAccessException or InvalidDataException)
        {
            _log.Error($"[UPDATE] Обновление прервано до установки: {ex.Message}");
            _log.Ok($"[UPDATE] Изменения на АРМ не вносились, работает версия {current.Version}");
            notes.Add($"Пакет отклонён: {ex.Message}. Установка не выполнялась.");
            await HealthAndActAsync($"Обновление {current.Version} → {release.Version} (прервано)",
current.Version,
                current.Version, null, rowsBefore: null, rolledBack: false, notes, started, ct, forceNotReady: true);
            return ExitCodes.Aborted;
        }

        // 3. Снимок данных и резервная копия
        Dictionary<string, long> rowsBefore;
        using (var db = LocalDb.Open(_layout.Database, readOnly: true))
            rowsBefore = LocalDb.CountRows(db, LocalDb.KeyTables);
        _log.Info($"[UPDATE] Снимок данных до обновления: {string.Join(", ", rowsBefore.Select(kv =>
$"{kv.Key}={kv.Value}"))}");

        var backups = new BackupService(_layout, _options.BackupRoot, _log, _options.BackupsToKeep);
        BackupInfo backup;
        try
        {
            backup = backups.Create(current.Version, activeSlot);
        }
        catch (Exception ex) when (ex is IOException or UnauthorizedAccessException or InvalidDataException or
Microsoft.Data.Sqlite.SqliteException)
        {
            _log.Error($"[UPDATE] Не удалось создать резервную копию: {ex.Message}. Обновление без копии
запрещено регламентом.");
            return ExitCodes.Aborted;
```

```
        }
        notes.Add($"Резервная копия данных: {Paths.Rel(backup.Path)}");

        // 4. Установка в неактивный слот + миграции
        var targetSlot = ArmLayout.OtherSlot(activeSlot);
        var installer = new BlueGreenInstaller(_layout, _log);
        AppVersionInfo installed;
        try
        {
            installed = installer.InstallToSlot(package, targetSlot);
            var schema = installer.ApplyMigrations(targetSlot);
            notes.Add($"Версия {installed.Version} установлена в слот '{targetSlot}', схема БД v{schema}");
        }
        catch (Exception ex)
        {
            // Активный слот не переключался — достаточно вернуть данные (миграция могла частично пройти).
            _log.Error($"[UPDATE] Ошибка установки: {ex.Message}");
            backups.Restore(backup);
            DiscardFailedSlot(targetSlot, release.Version);
            notes.Add($"Ошибка установки: {ex.Message}. Данные восстановлены, слот не переключался.");
            await HealthAndActAsync($"Обновление {current.Version} → {release.Version}", current.Version,
release.Version,
                backup.Path, rowsBefore, rolledBack: true, notes, started, ct);
            return ExitCodes.RolledBack;
        }

        // 5. Переключение Blue-Green
        _layout.SwitchActiveSlot(targetSlot);
        _log.Step($"[SWITCH] Активный слот: '{activeSlot}' → '{targetSlot}' (версия {installed.Version})");
        notes.Add($"Переключение слота {activeSlot} → {targetSlot}");

        // 6. Health Check новой версии
        var results = await RunChecksAsync(rowsBefore, ct);
        var failed = results.Where(r => r.Critical && r.Status == HealthStatus.Fail).ToList();

        if (failed.Count == 0)
        {
            _log.Ok($"[UPDATE] Обновление {current.Version} → {installed.Version} выполнено успешно");
            notes.Add("Все критичные проверки пройдены");
            Finish(new ReadinessReport
            {
                ArmId = _options.ArmId, Ring = _options.Ring,
                Operation = $"Обновление {current.Version} → {installed.Version}",
                VersionBefore = current.Version, VersionAfter = installed.Version,
                ActiveSlot = targetSlot, BackupPath = backup.Path,
                StartedAt = started, Results = results, Notes = notes
            });
            return ExitCodes.Ok;
        }

        // 7. Автоматический откат
        _log.Error($"[ROLLBACK] Провалены критичные проверки: {string.Join(", ", failed.Select(f => f.Code))}.
Запуск отката.");
        notes.Add($"Провалены проверки: {string.Join("; ", failed.Select(f => $"{f.Name} — {f.Details}"))}");

        _layout.SwitchActiveSlot(activeSlot);
        _log.Warn($"[ROLLBACK] Активный слот возвращён: '{targetSlot}' → '{activeSlot}' (версия
{current.Version})");
        backups.Restore(backup);
        notes.Add($"Откат: слот {targetSlot} → {activeSlot}, данные восстановлены из резервной копии");
        DiscardFailedSlot(targetSlot, installed.Version);

        _log.Step("[ROLLBACK] Контрольная проверка предыдущей версии после отката");
```

```
        var after = await RunChecksAsync(rowsBefore, ct);
        var restoredOk = after.All(r => !(r.Critical && r.Status == HealthStatus.Fail));
        notes.Add(restoredOk
            ? $"После отката версия {current.Version} прошла все критичные проверки"
            : "ВНИМАНИЕ: после отката есть непройденные проверки — требуется вмешательство
администратора");

        Finish(new ReadinessReport
        {
            ArmId = _options.ArmId, Ring = _options.Ring,
            Operation = $"Обновление {current.Version} → {installed.Version} (откат)",
            VersionBefore = current.Version, VersionAfter = installed.Version,
            ActiveSlot = activeSlot, BackupPath = backup.Path,
            StartedAt = started, Results = results.Concat(after.Select(r => r with { Name = r.Name + " (после
отката)" })).ToList(),
            Notes = notes, RolledBack = true
        });
        return ExitCodes.RolledBack;
    }

    // Команда healthcheck
    public async Task<int> HealthCheckAsync(CancellationToken ct = default)
    {
        var started = DateTime.Now;
        var slot = _layout.ReadActiveSlot();
        var version = _layout.ReadVersion(slot)?.Version ?? "не установлена";
        var report = await HealthAndActAsync("Плановая проверка готовности", version, version, null, null,
            rolledBack: false, [], started, ct);
        return report.Ready ? ExitCodes.Ok : ExitCodes.NotReady;
    }

    // Команда rollback (ручной откат администратором)
    public async Task<int> RollbackAsync(CancellationToken ct = default)
    {
        var started = DateTime.Now;
        var activeSlot = _layout.ReadActiveSlot();
        var targetSlot = ArmLayout.OtherSlot(activeSlot);
        var current = _layout.ReadVersion(activeSlot)
                ?? throw new InvalidDataException($"В активном слоте '{activeSlot}' нет установленной версии");

        // 1. Точка восстановления: последняя резервная копия, сделанная на версии ниже текущей
        var backup = FindRollbackPoint(current.Version);
        if (backup is null)
        {
            _log.Error($"[ROLLBACK] Нет резервной копии версии ниже {current.Version} — откат
невозможен");
            return ExitCodes.Aborted;
        }
        _log.Step($"[ROLLBACK] Ручной откат {current.Version} → {backup.Version}, точка восстановления
{Paths.Rel(backup.Path)}");
        var notes = new List<string> { $"Ручной откат {current.Version} → {backup.Version}" };

        // 2. Предыдущая версия должна лежать в резервном слоте; если её там нет — переустановить с сервера
        if (_layout.ReadVersion(targetSlot)?.Version != backup.Version)
        {
            try
            {
                var release = LoadManifest().Releases.FirstOrDefault(r => r.Version == backup.Version)
                        ?? throw new InvalidDataException($"пакет версии {backup.Version} отсутствует на сервере
обновлений");
                new BlueGreenInstaller(_layout, _log).InstallToSlot(Download(release), targetSlot);
                notes.Add($"Версия {backup.Version} переустановлена в слот '{targetSlot}' с сервера обновлений");
            }
```

```
        catch (Exception ex) when (ex is IOException or UnauthorizedAccessException or InvalidDataException or
JsonException)
        {
            _log.Error($"[ROLLBACK] Не удалось подготовить версию {backup.Version}: {ex.Message}.
Работает версия {current.Version}.");
            return ExitCodes.Aborted;
        }
    }

    // 3. Переключение слота и восстановление данных
    _layout.SwitchActiveSlot(targetSlot);
    _log.Warn($"[ROLLBACK] Активный слот: '{activeSlot}' ({current.Version}) → '{targetSlot}'
({backup.Version})");
    new BackupService(_layout, _options.BackupRoot, _log, _options.BackupsToKeep).Restore(backup);
    notes.Add($"Данные восстановлены из {Paths.Rel(backup.Path)}");

    // 4. Контрольная проверка и акт
    var report = await HealthAndActAsync($"Ручной откат {current.Version} → {backup.Version}",
current.Version,
        backup.Version, backup.Path, null, rolledBack: false, notes, started, ct);
    return report.Ready ? ExitCodes.Ok : ExitCodes.NotReady;
  }

  // Команды status / check
  public int Status()
  {
    var slot = _layout.ReadActiveSlot();
    var v = _layout.ReadVersion(slot);
    var other = _layout.ReadVersion(ArmLayout.OtherSlot(slot));
    _log.Step($"[STATUS] APM {_options.ArmId}, кольцо '{_options.Ring}'");
    _log.Info($"[STATUS] Активный слот '{slot}': {v?.Version ?? "пусто"}; резервный слот
'{ArmLayout.OtherSlot(slot)}': {other?.Version ?? "пусто"}");
    using var db = LocalDb.Open(_layout.Database, readOnly: true);
    _log.Info($"[STATUS] Схема БД v{LocalDb.GetSchemaVersion(db)}; " +
          string.Join(", ", LocalDb.CountRows(db, LocalDb.KeyTables).Select(kv =>
$"{kv.Key}={kv.Value}")));
    return ExitCodes.Ok;
  }

  public int Check()
  {
    var current = _layout.ReadVersion(_layout.ReadActiveSlot());
    ReleaseManifest manifest;
    try { manifest = LoadManifest(); }
    catch (Exception ex) when (ex is IOException or UnauthorizedAccessException or JsonException)
    {
      _log.Error($"[CHECK] Сервер обновлений недоступен: {ex.Message}");
      return ExitCodes.Aborted;
    }

    _log.Step($"[CHECK] Релизы на сервере (текущая версия {current?.Version}, кольцо '{_options.Ring}'):");
    foreach (var r in manifest.Releases.OrderBy(r => r.ParsedVersion))
    {
      var available = r.Rings.Contains(_options.Ring, StringComparer.OrdinalIgnoreCase);
      _log.Info($"  {r.Version,-8} кольца [{string.Join(",", r.Rings)}] {(available ? "доступен" : "НЕ доступен
для этого кольца")} — {r.Notes}");
    }
    return ExitCodes.Ok;
  }

  // Вспомогательные методы
  private ReleaseManifest LoadManifest()
  {
```

```
        var file = Path.Combine(_options.UpdateSource, "manifest.json");
        var text = Retry.Io(() => File.ReadAllText(file), _log, "Чтение манифеста");
        return JsonSerializer.Deserialize<ReleaseManifest>(text, Json.Options)
            ?? throw new InvalidDataException("Пустой манифест сервера обновлений");
    }

    /// <summary>Выбор релиза с учётом кольца развёртывания (Canary Deployment).</summary>
    private ReleaseInfo? SelectRelease(ReleaseManifest manifest, AppVersionInfo current, string? targetVersion)
    {
        var currentVersion = Version.Parse(current.Version);
        var forRing = manifest.Releases
            .Where(r => r.Rings.Contains(_options.Ring, StringComparer.OrdinalIgnoreCase))
            .ToList();

        ReleaseInfo? release;
        if (targetVersion is not null)
        {
            release = manifest.Releases.FirstOrDefault(r => r.Version == targetVersion);
            if (release is null)
            {
                _log.Warn($"[UPDATE] Релиз {targetVersion} на сервере не найден");
                return null;
            }
            if (!forRing.Contains(release))
            {
                _log.Warn($"[UPDATE] Релиз {targetVersion} ещё не доступен кольцу '{_options.Ring}' (только
[{string.Join(",", release.Rings)}])");
                return null;
            }
        }
        else
        {
            release = forRing.Where(r => r.ParsedVersion > currentVersion).MaxBy(r => r.ParsedVersion);
        }

        if (release is null || release.ParsedVersion <= currentVersion)
        {
            _log.Ok($"[UPDATE] Обновлений нет: установлена актуальная версия {current.Version}");
            return null;
        }
        return release;
    }

    /// <summary>«Загрузка» пакета с сервера (копирование из сетевой папки) и проверка
SHA-256.</summary>
    private string Download(ReleaseInfo release)
    {
        var source = Path.Combine(_options.UpdateSource, release.Package);
        Directory.CreateDirectory(_layout.DownloadsDir);
        var target = Path.Combine(_layout.DownloadsDir, Path.GetFileName(release.Package));

        Retry.Io(() => { File.Copy(source, target, overwrite: true); return 0; }, _log, "Загрузка пакета");
        var size = new FileInfo(target).Length;
        var hash = Hashing.Sha256File(target);
        _log.Info($"[DOWNLOAD] {Path.GetFileName(target)}: {size / 1024.0:0.0} КБ, SHA-256 {hash[..16]}…");

        if (!hash.Equals(release.Sha256, StringComparison.OrdinalIgnoreCase))
        {
            File.Delete(target);
            throw new InvalidDataException($"контрольная сумма пакета {release.Version} не совпадает с
манифестом " +
                                           $"(ожидалось {release.Sha256[..16]}…, получено {hash[..16]}…) — пакет повреждён
или подменён");
```

```
        }
        _log.Ok("[DOWNLOAD] Контрольная сумма пакета совпадает с манифестом");
        return target;
    }

    /// <summary>Последняя проверенная резервная копия, сделанная на версии ниже текущей.</summary>
    private BackupInfo? FindRollbackPoint(string currentVersion)
    {
        if (!Directory.Exists(_options.BackupRoot)) return null;
        var current = Version.Parse(currentVersion);
        return new DirectoryInfo(_options.BackupRoot).GetDirectories()
            .OrderByDescending(d => d.Name, StringComparer.Ordinal)
            .Select(d => Path.Combine(d.FullName, "backup.json"))
            .Where(File.Exists)
            .Select(f => JsonSerializer.Deserialize<BackupInfo>(File.ReadAllText(f), Json.Options))
            .FirstOrDefault(b => b is not null && Version.TryParse(b.Version, out var v) && v < current);
    }

    /// <summary>Слот с версией, не прошедшей проверку, очищается — её нельзя случайно «вернуть»
ручным откатом.</summary>
    private void DiscardFailedSlot(string slot, string version)
    {
        try
        {
            Directory.Delete(_layout.SlotDir(slot), recursive: true);
            _log.Info($"[ROLLBACK] Слот '{slot}' с непроверенной версией {version} очищен");
        }
        catch (Exception ex) when (ex is IOException or UnauthorizedAccessException)
        {
            _log.Warn($"[ROLLBACK] Не удалось очистить слот '{slot}': {ex.Message}");
        }
    }

    private IReadOnlyList<IHealthCheck> BuildChecks() =>
    [
        new IntegrityCheck(),
        new DatabaseCheck(),
        new ApiCheck(_apiHandler),
        new ReaderCheck(ReaderCheck.Create(_options.HealthCheck.Reader, _layout.ReaderTestCard))
    ];

    private Task<IReadOnlyList<HealthCheckResult>> RunChecksAsync(IReadOnlyDictionary<string, long>?
rowsBefore, CancellationToken ct) =>
        new HealthCheckRunner(BuildChecks()).RunAsync(new HealthContext
        {
            Layout = _layout,
            Options = _options,
            Log = _log,
            RowsBefore = rowsBefore
        }, ct);

    private async Task<ReadinessReport> HealthAndActAsync(string operation, string before, string after, string?
backupPath,
        IReadOnlyDictionary<string, long>? rowsBefore, bool rolledBack, List<string> notes, DateTime started,
        CancellationToken ct, bool forceNotReady = false)
    {
        var results = await RunChecksAsync(rowsBefore, ct);
        if (forceNotReady)
            results = results.Append(new HealthCheckResult("PACKAGE", "Пакет обновления", HealthStatus.Fail, 0,
                "пакет отклонён, обновление не установлено", true)).ToList();

        var report = new ReadinessReport
        {
```

```
            ArmId = _options.ArmId, Ring = _options.Ring, Operation = operation,
            VersionBefore = before, VersionAfter = after,
            ActiveSlot = _layout.ReadActiveSlot(), BackupPath = backupPath,
            StartedAt = started, Results = results, Notes = notes, RolledBack = rolledBack
        };
        Finish(report);
        return report;
    }

    private void Finish(ReadinessReport report)
    {
        ReadinessAct.PrintSummary(report);
        try
        {
            LastActPath = ReadinessAct.Write(report, _options.ReportsRoot);
            _log.Ok($"[ACT] Акт готовности сохранён: {Paths.Rel(LastActPath)}");
        }
        catch (Exception ex) when (ex is IOException or UnauthorizedAccessException)
        {
            _log.Error($"[ACT] Не удалось сохранить акт: {ex.Message}");
        }
        if (_log.FilePath is not null) _log.Info($"[LOG] Журнал: {Paths.Rel(_log.FilePath)}");
    }
}
```

Листинг А.4 – Файл Campus.Arm.Updater/Update/BackupService.cs

```
using System.Text.Json;
using Campus.Arm.Updater.Infrastructure;
using Microsoft.Data.Sqlite;

namespace Campus.Arm.Updater.Update;

/// <summary>Сведения о резервной копии (backup.json внутри каталога копии).</summary>
public sealed class BackupInfo
{
    public string Path { get; set; } = "";
    public string Version { get; set; } = "";
    public string ActiveSlot { get; set; } = "";
    public DateTime CreatedAt { get; set; }

    /// <summary>Относительный путь файла → SHA-256 (для проверки копии перед
восстановлением).</summary>
    public Dictionary<string, string> Files { get; set; } = new();
}

/// <summary>
/// Резервное копирование локальных данных АРМ перед обновлением:
/// appsettings.json, кэш справочников и локальная БД SQLite.
/// БД копируется через онлайн-API SQLite (BackupDatabase) — получается согласованная копия,
/// даже если клиент АРМ в этот момент пишет в журнал проходов.
/// </summary>
public sealed class BackupService(ArmLayout layout, string backupRoot, AppLogger log, int keep = 5)
{
    public BackupInfo Create(string version, string activeSlot)
    {
        var dir = Path.Combine(backupRoot, $"{DateTime.Now:yyyyMMdd_HHmmss_fff}_v{version}");
        Directory.CreateDirectory(dir);
        log.Info($"[BACKUP] Каталог копии: {Paths.Rel(dir)}");

        // 1. Конфигурация АРМ
        Retry.Io(() => { File.Copy(layout.AppSettings, Path.Combine(dir, "appsettings.json")); return 0; },
```

```
            log, "Копирование appsettings.json");

        // 2. Кэш справочников
        CopyDirectory(layout.CacheDir, Path.Combine(dir, "cache"));

        // 3. Локальная БД — онлайн-копия средствами SQLite
        using (var source = LocalDb.Open(layout.Database, readOnly: true))
        using (var target = new SqliteConnection($"Data Source={Path.Combine(dir, "campus-
local.db")};Pooling=False"))
        {
            target.Open();
            source.BackupDatabase(target);
        }

        // 4. Манифест копии с контрольными суммами
        var info = new BackupInfo
        {
            Path = dir,
            Version = version,
            ActiveSlot = activeSlot,
            CreatedAt = DateTime.Now
        };
        foreach (var file in Directory.EnumerateFiles(dir, "*", SearchOption.AllDirectories))
            info.Files[System.IO.Path.GetRelativePath(dir, file)] = Hashing.Sha256File(file);

        File.WriteAllText(Path.Combine(dir, "backup.json"), JsonSerializer.Serialize(info, Json.Options));

        Verify(info);
        log.Ok($"[BACKUP] Резервная копия создана и проверена: {info.Files.Count} файлов (appsettings, кэш,
SQLite)");

        Rotate();
        return info;
    }

    /// <summary>Сверка контрольных сумм копии. Копию, не прошедшую проверку, восстанавливать
нельзя.</summary>
    public void Verify(BackupInfo info)
    {
        foreach (var (relative, expected) in info.Files)
        {
            var actual = Hashing.Sha256File(Path.Combine(info.Path, relative));
            if (actual != expected)
                throw new InvalidDataException($"Резервная копия повреждена: {relative} (SHA-256 не
совпадает)");
        }
    }

    /// <summary>Восстановление локальных данных из резервной копии (часть плана отката).</summary>
    public void Restore(BackupInfo info)
    {
        Verify(info);
        log.Warn($"[ROLLBACK] Восстановление данных из {Paths.Rel(info.Path)}");

        SqliteConnection.ClearAllPools();

        Retry.Io(() => { File.Copy(Path.Combine(info.Path, "appsettings.json"), layout.AppSettings, overwrite: true);
return 0; },
            log, "Восстановление appsettings.json");

        if (Directory.Exists(layout.CacheDir)) Directory.Delete(layout.CacheDir, recursive: true);
        CopyDirectory(Path.Combine(info.Path, "cache"), layout.CacheDir);
```

```
        // Служебные файлы журнала SQLite от новой версии не должны «накатиться» на старую БД
        foreach (var suffix in new[] { "-wal", "-shm", "-journal" })
            if (File.Exists(layout.Database + suffix)) File.Delete(layout.Database + suffix);

        Retry.Io(() => { File.Copy(Path.Combine(info.Path, "campus-local.db"), layout.Database, overwrite: true);
return 0; },
            log, "Восстановление campus-local.db");

        log.Ok("[ROLLBACK] Данные восстановлены: appsettings.json, кэш, локальная БД");
    }

    /// <summary>Хранить только последние N копий.</summary>
    private void Rotate()
    {
        var old = new DirectoryInfo(backupRoot).GetDirectories()
            .OrderByDescending(d => d.Name, StringComparer.Ordinal)
            .Skip(keep);
        foreach (var dir in old)
        {
            try
            {
                dir.Delete(recursive: true);
                log.Info($"[BACKUP] Удалена устаревшая копия {dir.Name}");
            }
            catch (Exception ex) when (ex is IOException or UnauthorizedAccessException)
            {
                log.Warn($"[BACKUP] Не удалось удалить старую копию {dir.Name}: {ex.Message}");
            }
        }
    }

    private static void CopyDirectory(string from, string to)
    {
        Directory.CreateDirectory(to);
        if (!Directory.Exists(from)) return;
        foreach (var file in Directory.EnumerateFiles(from, "*", SearchOption.AllDirectories))
        {
            var target = Path.Combine(to, Path.GetRelativePath(from, file));
            Directory.CreateDirectory(Path.GetDirectoryName(target)!);
            File.Copy(file, target, overwrite: true);
        }
    }
}
```

Листинг А.5 – Файл Campus.Arm.Updater/Update/BlueGreenInstaller.cs

```
using System.IO.Compression;
using Campus.Arm.Updater.Infrastructure;

namespace Campus.Arm.Updater.Update;

/// <summary>
/// Установка новой версии в неактивный слот (Blue-Green) и применение миграций локальной БД.
/// Работающая версия в активном слоте не затрагивается, пока новая не проверена.
/// </summary>
public sealed class BlueGreenInstaller(ArmLayout layout, AppLogger log)
{
    /// <summary>Распаковка пакета в слот и сверка SHA-256 каждого файла с version.json.</summary>
    public AppVersionInfo InstallToSlot(string packageZip, string slot)
    {
        var dir = layout.SlotDir(slot);
        if (Directory.Exists(dir)) Directory.Delete(dir, recursive: true);
```

```
        Directory.CreateDirectory(dir);

        try
        {
            ZipFile.ExtractToDirectory(packageZip, dir);
        }
        catch (InvalidDataException ex)
        {
            throw new InvalidDataException($"Пакет {Path.GetFileName(packageZip)} не является корректным zip-
архивом: {ex.Message}", ex);
        }

        var info = layout.ReadVersion(slot)
                ?? throw new InvalidDataException("В пакете отсутствует version.json");

        foreach (var (relative, expected) in info.Files)
        {
            var file = Path.Combine(dir, relative.Replace('/', Path.DirectorySeparatorChar));
            if (!File.Exists(file))
                throw new InvalidDataException($"В пакете отсутствует файл {relative}");
            if (Hashing.Sha256File(file) != expected)
                throw new InvalidDataException($"Файл {relative} повреждён (SHA-256 не совпадает)");
        }

        log.Ok($"[INSTALL] Версия {info.Version} распакована в слот '{slot}', проверено файлов:
{info.Files.Count}");
        return info;
    }

    /// <summary>
    /// Применение SQL-миграций из папки migrations/ пакета. Каждая миграция — в своей транзакции;
    /// номер миграции берётся из имени файла (003_add_reader_zone.sql → 3).
    /// </summary>
    public int ApplyMigrations(string slot)
    {
        var migrationsDir = Path.Combine(layout.SlotDir(slot), "migrations");
        using var connection = LocalDb.Open(layout.Database);
        var current = LocalDb.GetSchemaVersion(connection);

        if (!Directory.Exists(migrationsDir)) return current;

        foreach (var file in Directory.GetFiles(migrationsDir, "*.sql").OrderBy(f => f, StringComparer.Ordinal))
        {
            var name = Path.GetFileName(file);
            if (!int.TryParse(name.Split('_')[0], out var number))
                throw new InvalidDataException($"Некорректное имя миграции: {name}");
            if (number <= current) continue;

            using var tx = connection.BeginTransaction();
            using var cmd = connection.CreateCommand();
            cmd.Transaction = tx;
            cmd.CommandText = File.ReadAllText(file) + $";\nUPDATE schema_info SET version = {number};";
            cmd.ExecuteNonQuery();
            tx.Commit();

            current = number;
            log.Info($"[MIGRATE] Применена миграция {name}, версия схемы БД = {current}");
        }
        return current;
    }
}
```

Листинг А.6 – Файл Campus.Arm.Updater/Update/ArmLayout.cs

```
using System.Text.Json;
using System.Text.Json.Serialization;
using Campus.Arm.Updater.Infrastructure;

namespace Campus.Arm.Updater.Update;

/// <summary>
/// Структура каталога клиента АРМ «Умный кампус» (схема Blue-Green):
/// <code>
/// InstallRoot/
///   slots/blue/           ← версия N   (приложение, version.json)
///   slots/green/          ← версия N+1 (ставится «рядом», пока работает N)
///   active-slot.txt       ← какой слот сейчас запускается ("blue"/"green")
///   data/appsettings.json ← локальные настройки АРМ
///   data/cache/           ← кэш справочников
///   data/campus-local.db  ← локальная БД SQLite (журнал проходов, очередь синхронизации)
/// </code>
/// Переключение версии = атомарная замена active-slot.txt, откат = переключение обратно.
/// </summary>
public sealed class ArmLayout(string root)
{
    public string Root { get; } = root;
    public string SlotsDir => Path.Combine(Root, "slots");
    public string ActiveSlotFile => Path.Combine(Root, "active-slot.txt");
    public string DataDir => Path.Combine(Root, "data");
    public string AppSettings => Path.Combine(DataDir, "appsettings.json");
    public string CacheDir => Path.Combine(DataDir, "cache");
    public string Database => Path.Combine(DataDir, "campus-local.db");
    public string DownloadsDir => Path.Combine(Root, "downloads");
    public string ReaderTestCard => Path.Combine(Root, "reader", "test-card.txt");

    public string SlotDir(string slot) => Path.Combine(SlotsDir, slot);

    public static string OtherSlot(string slot) => slot == "blue" ? "green" : "blue";

    public string ReadActiveSlot()
    {
        if (!File.Exists(ActiveSlotFile)) return "blue";
        var slot = File.ReadAllText(ActiveSlotFile).Trim();
        return slot is "blue" or "green"
            ? slot
            : throw new InvalidDataException($"Повреждён файл {ActiveSlotFile}: '{slot}'");
    }

    /// <summary>Атомарное переключение слота: запись во временный файл и замена.</summary>
    public void SwitchActiveSlot(string slot)
    {
        var tmp = ActiveSlotFile + ".tmp";
        File.WriteAllText(tmp, slot);
        File.Move(tmp, ActiveSlotFile, overwrite: true);
    }

    /// <summary>Чтение version.json из слота (null — слот пуст).</summary>
    public AppVersionInfo? ReadVersion(string slot)
    {
        var file = Path.Combine(SlotDir(slot), "version.json");
        if (!File.Exists(file)) return null;
        try
        {
            return JsonSerializer.Deserialize<AppVersionInfo>(File.ReadAllText(file), Json.Options);
        }
```

```
        catch (JsonException ex)
        {
            throw new InvalidDataException($"Повреждён {file}: {ex.Message}", ex);
        }
    }
}

/// <summary>Описание версии клиента АРМ (version.json внутри пакета обновления).</summary>
public sealed class AppVersionInfo
{
    public string Version { get; set; } = "0.0.0";

    /// <summary>Версия схемы локальной БД, которую ожидает эта версия клиента.</summary>
    public int SchemaVersion { get; set; }

    /// <summary>Колонки, без которых клиент не работает (формат "таблица.колонка").</summary>
    public List<string> RequiredColumns { get; set; } = [];

    /// <summary>Контрольные суммы файлов пакета (путь → SHA-256).</summary>
    public Dictionary<string, string> Files { get; set; } = new();

    public DateTime BuildDate { get; set; }
}

/// <summary>Манифест сервера обновлений (manifest.json в папке UpdateSource).</summary>
public sealed class ReleaseManifest
{
    public string Product { get; set; } = "Campus.Arm.Client";
    public List<ReleaseInfo> Releases { get; set; } = [];
}

public sealed class ReleaseInfo
{
    public string Version { get; set; } = "";

    /// <summary>Кольца, которым доступен релиз: ["canary"] — опытная эксплуатация, ["canary","stable"] — промышленная.</summary>
    public List<string> Rings { get; set; } = [];

    /// <summary>Путь к zip-пакету относительно папки сервера обновлений.</summary>
    public string Package { get; set; } = "";

    /// <summary>SHA-256 zip-пакета — проверяется до установки.</summary>
    public string Sha256 { get; set; } = "";

    public string Notes { get; set; } = "";
    public DateTime Published { get; set; }

    [JsonIgnore]
    public System.Version ParsedVersion => System.Version.Parse(Version);
}
```

Листинг А.7 – Файл Campus.Arm.Updater/Health/HealthModel.cs

```
using Campus.Arm.Updater.Config;
using Campus.Arm.Updater.Infrastructure;
using Campus.Arm.Updater.Update;

namespace Campus.Arm.Updater.Health;

public enum HealthStatus
{
```

```
    Pass,   // проверка пройдена
    Warn,   // пройдена, но задержка выше порога
    Fail    // не пройдена
}

/// <summary>Результат одной проверки цепочки Health Check.</summary>
public sealed record HealthCheckResult(
    string Code,
    string Name,
    HealthStatus Status,
    double LatencyMs,
    string Details,
    bool Critical)
{
    public string StatusText => Status switch
    {
        HealthStatus.Pass => "ПРОЙДЕНА",
        HealthStatus.Warn => "ПРЕДУПРЕЖДЕНИЕ",
        _ => "НЕ ПРОЙДЕНА"
    };
}

/// <summary>Данные, общие для всех проверок одного прогона.</summary>
public sealed class HealthContext
{
    public required ArmLayout Layout { get; init; }
    public required UpdaterOptions Options { get; init; }
    public required AppLogger Log { get; init; }

    /// <summary>Количество строк в ключевых таблицах до обновления (для проверки сохранности
данных).</summary>
    public IReadOnlyDictionary<string, long>? RowsBefore { get; init; }
}

/// <summary>Одна проверка в цепочке.</summary>
public interface IHealthCheck
{
    /// <summary>Короткий код для журнала: INTEGRITY, DB, API, READER.</summary>
    string Code { get; }
    string Name { get; }

    /// <summary>Критичная проверка: её провал после обновления запускает откат.</summary>
    bool Critical { get; }

    /// <summary>Порог задержки, мс (0 — без порога).</summary>
    int WarnLatencyMs(HealthCheckOptions options);

    /// <summary>Выполнить проверку. Возвращает текст результата; при ошибке бросает исключение
    /// (его перехватывает <see cref="HealthCheckRunner"/> и превращает в статус «НЕ
ПРОЙДЕНА»).</summary>
    Task<string> RunAsync(HealthContext context, CancellationToken ct);
}
```

Листинг А.8 – Файл Campus.Arm.Updater/Health/HealthCheckRunner.cs

```
using System.Diagnostics;
using Campus.Arm.Updater.Infrastructure;
using Microsoft.Data.Sqlite;

namespace Campus.Arm.Updater.Health;

/// <summary>
```

```
/// Выполняет цепочку проверок по порядку (целостность → БД → API → считыватель),
/// измеряет задержку (Latency) каждой проверки и пишет её в журнал.
/// Любое исключение внутри проверки превращается в результат «НЕ ПРОЙДЕНА»,
/// поэтому один сбой не обрывает всю цепочку и акт готовности всегда формируется.
/// </summary>
public sealed class HealthCheckRunner(IReadOnlyList<IHealthCheck> checks)
{
    public async Task<IReadOnlyList<HealthCheckResult>> RunAsync(HealthContext context, CancellationToken
ct = default)
    {
        var log = context.Log;
        var results = new List<HealthCheckResult>();
        log.Step($"[HEALTH] Запуск цепочки проверок ({checks.Count}): {string.Join(" → ", checks.Select(c =>
c.Code))}");

        foreach (var check in checks)
        {
            var sw = Stopwatch.StartNew();
            HealthStatus status;
            string details;
            try
            {
                details = await check.RunAsync(context, ct);
                status = HealthStatus.Pass;
            }
            catch (OperationCanceledException) when (ct.IsCancellationRequested)
            {
                throw; // прервано пользователем (Ctrl+C)
            }
            catch (Exception ex)
            {
                status = HealthStatus.Fail;
                details = Describe(ex);
            }
            sw.Stop();

            var latency = Math.Round(sw.Elapsed.TotalMilliseconds, 1);
            var warnMs = check.WarnLatencyMs(context.Options.HealthCheck);
            if (status == HealthStatus.Pass && warnMs > 0 && latency > warnMs)
            {
                status = HealthStatus.Warn;
                details += $" (задержка {latency} мс > порога {warnMs} мс)";
            }

            var result = new HealthCheckResult(check.Code, check.Name, status, latency, details, check.Critical);
            results.Add(result);

            // Строка 1 — код, статус и задержка; строка 2 — подробности проверки
            Action<string> write = status switch
            {
                HealthStatus.Pass => log.Ok,
                HealthStatus.Warn => log.Warn,
                _ => log.Error
            };
            write($"[HEALTH] {check.Code,-9} {result.StatusText,-14} latency={latency,8:0.0} мс");
            write($"[HEALTH]    └─ {details}");
        }
        return results;
    }

    /// <summary>Понятное описание типовых исключений для журнала и акта.</summary>
    private static string Describe(Exception ex) => ex switch
    {
```

```
        FileNotFoundException f => $"файл не найден: {f.FileName ?? f.Message}",
        UnauthorizedAccessException u => $"нет доступа: {u.Message}",
        TimeoutException t => $"таймаут: {t.Message}",
        SqliteException s => $"ошибка SQLite {s.SqliteErrorCode}: {s.Message}",
        HttpRequestException h => $"сетевая ошибка: {h.Message}",
        IOException io => $"ошибка ввода-вывода: {io.Message}",
        _ => ex.Message
    };
}
```

Листинг A.9 – Файл Campus.Arm.Updater/Health/Checks.cs

```
using System.Diagnostics;
using System.Net;
using System.Text.Json;
using System.Text.RegularExpressions;
using Campus.Arm.Updater.Config;
using Campus.Arm.Updater.Infrastructure;

namespace Campus.Arm.Updater.Health;

// Проверка 1. Целостность установки и данных

/// <summary>
/// Проверяет, что активный слот содержит все файлы пакета без искажений,
/// appsettings.json читается и содержит обязательные ключи,
/// а количество записей в локальной БД не уменьшилось после обновления.
/// </summary>
public sealed class IntegrityCheck : IHealthCheck
{
    private static readonly string[] RequiredSettings = ["ArmId", "CentralApiUrl", "ReaderMode"];

    public string Code => "INTEGRITY";
    public string Name => "Целостность установки и данных";
    public bool Critical => true;
    public int WarnLatencyMs(HealthCheckOptions o) => 0;

    public Task<string> RunAsync(HealthContext ctx, CancellationToken ct)
    {
        var slot = ctx.Layout.ReadActiveSlot();
        var version = ctx.Layout.ReadVersion(slot)
                      ?? throw new InvalidDataException($"В активном слоте '{slot}' нет version.json");

        // Файлы приложения
        foreach (var (relative, expected) in version.Files)
        {
            var file = Path.Combine(ctx.Layout.SlotDir(slot), relative.Replace('/', Path.DirectorySeparatorChar));
            if (!File.Exists(file)) throw new FileNotFoundException("Отсутствует файл приложения", relative);
            if (Hashing.Sha256File(file) != expected)
                throw new InvalidDataException($"Файл {relative} изменён (SHA-256 не совпадает)");
        }

        // Конфигурация АРМ
        using (var doc = JsonDocument.Parse(File.ReadAllText(ctx.Layout.AppSettings)))
        {
            foreach (var key in RequiredSettings)
                if (!doc.RootElement.TryGetProperty(key, out _))
                    throw new InvalidDataException($"В appsettings.json нет обязательного ключа '{key}'");
        }

        // Сохранность данных: записей не меньше, чем до обновления
        using var db = LocalDb.Open(ctx.Layout.Database, readOnly: true);
```

```
        var rowsNow = LocalDb.CountRows(db, LocalDb.KeyTables);
        if (ctx.RowsBefore is not null)
        {
            foreach (var (table, before) in ctx.RowsBefore)
                if (rowsNow.TryGetValue(table, out var now) && now < before)
                    throw new InvalidDataException($"Потеря данных в таблице {table}: было {before}, стало
{now}");
        }

        var rows = string.Join(", ", rowsNow.Select(kv => $"{kv.Key}={kv.Value}"));
        return Task.FromResult($"v{version.Version}, слот {slot}, файлов {version.Files.Count}; {rows}");
    }
}

// Проверка 2. Локальная БД SQLite

/// <summary>
/// PRAGMA integrity_check, соответствие версии схемы и обязательных колонок версии клиента,
/// тестовая запись в транзакции с откатом (БД доступна на запись).
/// </summary>
public sealed class DatabaseCheck : IHealthCheck
{
    public string Code => "DB";
    public string Name => "Локальная БД SQLite";
    public bool Critical => true;
    public int WarnLatencyMs(HealthCheckOptions o) => o.DatabaseWarnMs;

    public Task<string> RunAsync(HealthContext ctx, CancellationToken ct)
    {
        var version = ctx.Layout.ReadVersion(ctx.Layout.ReadActiveSlot())
                      ?? throw new InvalidDataException("Не удалось определить версию клиента");

        using var db = LocalDb.Open(ctx.Layout.Database);

        var integrity = LocalDb.Scalar(db, "PRAGMA integrity_check")?.ToString();
        if (integrity != "ok")
            throw new InvalidDataException($"PRAGMA integrity_check: {integrity}");

        var schema = LocalDb.GetSchemaVersion(db);
        if (schema != version.SchemaVersion)
            throw new InvalidDataException($"версия схемы БД {schema}, клиент v{version.Version} ожидает
{version.SchemaVersion}");

        foreach (var required in version.RequiredColumns)
        {
            var parts = required.Split('.');
            if (parts.Length != 2 || !LocalDb.ColumnExists(db, parts[0], parts[1]))
                throw new InvalidDataException($"в БД нет колонки {required}, необходимой версии
{version.Version}");
        }

        // Пробная запись: БД не только читается, но и принимает новые события прохода
        using (var tx = db.BeginTransaction())
        {
            using var cmd = db.CreateCommand();
            cmd.Transaction = tx;
            cmd.CommandText = "INSERT INTO access_events (card_uid, direction, event_time) VALUES
('HEALTHCHECK', 'IN', datetime('now'))";
            cmd.ExecuteNonQuery();
            tx.Rollback();
        }

        var events = LocalDb.Scalar(db, "SELECT COUNT(*) FROM access_events");
```

```
        return Task.FromResult($"integrity_check=ok, схема v{schema}, событий {events}, запись OK");
    }
}

// Проверка 3. Центральный API (по варианту — https://api.ipify.org?format=json)

/// <summary>
/// HTTP-запрос к API с таймаутом на каждую попытку и повторами с экспоненциальной паузой.
/// Обрабатываются: таймаут (network timeout), ошибки соединения/DNS, HTTP-коды ошибок,
/// некорректный JSON. Внешний IP в журнале маскируется.
/// </summary>
public sealed class ApiCheck(Func<HttpMessageHandler>? handlerFactory = null) : IHealthCheck
{
    public string Code => "API";
    public string Name => "Центральный API";
    public bool Critical => true;
    public int WarnLatencyMs(HealthCheckOptions o) => o.ApiWarnMs;

    public async Task<string> RunAsync(HealthContext ctx, CancellationToken ct)
    {
        var o = ctx.Options.HealthCheck;
        var attempts = Math.Max(1, o.ApiRetries);

        using var http = handlerFactory is null ? new HttpClient() : new HttpClient(handlerFactory(), disposeHandler:
true);
        http.Timeout = Timeout.InfiniteTimeSpan;            // таймаут задаём на каждую попытку через
CancellationToken
        http.DefaultRequestHeaders.UserAgent.ParseAdd("Campus.Arm.Updater/1.0");

        var lastError = "";
        for (var attempt = 1; attempt <= attempts; attempt++)
        {
            using var cts = CancellationTokenSource.CreateLinkedTokenSource(ct);
            cts.CancelAfter(TimeSpan.FromSeconds(o.ApiTimeoutSeconds));
            var sw = Stopwatch.StartNew();
            try
            {
                using var response = await http.GetAsync(o.ApiUrl, cts.Token);
                var body = await response.Content.ReadAsStringAsync(cts.Token);
                sw.Stop();

                if (!response.IsSuccessStatusCode)
                    throw new HttpRequestException($"HTTP {(int)response.StatusCode} {response.ReasonPhrase}");

                using var doc = JsonDocument.Parse(body);
                var ip = doc.RootElement.GetProperty("ip").GetString();
                if (!IPAddress.TryParse(ip, out _))
                    throw new FormatException($"ответ не содержит корректный IP: '{ip}'");

                ctx.Log.Info($"[API] попытка {attempt}/{attempts}: HTTP {(int)response.StatusCode},
{sw.ElapsedMilliseconds} мс, внешний IP {Mask(ip!)}");
                return $"GET {o.ApiUrl} → 200 OK, {sw.ElapsedMilliseconds} мс (попытка {attempt}/{attempts}), IP
{Mask(ip!)}";
            }
            catch (OperationCanceledException) when (!ct.IsCancellationRequested)
            {
                lastError = $"таймаут {o.ApiTimeoutSeconds} с";
            }
            catch (HttpRequestException ex)
            {
                lastError = $"сетевая ошибка: {ex.Message}";
            }
```

```
        catch (Exception ex) when (ex is JsonException or KeyNotFoundException or FormatException or
InvalidOperationException)
        {
            lastError = $"некорректный ответ API: {ex.Message}";
        }

        ctx.Log.Warn($"[API] попытка {attempt}/{attempts} неудачна через {sw.ElapsedMilliseconds} мс:
{lastError}");
        if (attempt < attempts)
        {
            var pause = TimeSpan.FromMilliseconds(500 * Math.Pow(2, attempt - 1));
            ctx.Log.Info($"[API] повтор через {pause.TotalMilliseconds} мс");
            await Task.Delay(pause, ct);
        }
      }

      throw new HttpRequestException($"API недоступен после {attempts} попыток ({lastError})");
   }

   /// <summary>Маскирование IP: 95.24.13.7 → 95.24.*.* (адрес не попадает в отчёты целиком).</summary>
   internal static string Mask(string ip)
   {
      var parts = ip.Split('.');
      if (parts.Length == 4) return $"{parts[0]}.{parts[1]}.*.*";
      var v6 = ip.Split(':');
      return v6.Length > 2 ? $"{v6[0]}:{v6[1]}:*" : "*";
   }
}

// Проверка 4. Считыватель карт

/// <summary>Абстракция считывателя пропусков/студенческих билетов.</summary>
public interface ICardReader
{
   string Description { get; }
   Task<string> ReadCardAsync(TimeSpan timeout, CancellationToken ct);
}

/// <summary>Эмулятор считывателя для стенда опытной эксплуатации: UID тестовой карты берётся из
файла.</summary>
public sealed class EmulatedCardReader(string testCardFile) : ICardReader
{
   public string Description => "эмулятор";

   public async Task<string> ReadCardAsync(TimeSpan timeout, CancellationToken ct)
   {
      if (!File.Exists(testCardFile))
         throw new FileNotFoundException("Файл тестовой карты эмулятора не найден", testCardFile);
      await Task.Delay(40, ct);              // имитация времени чтения карты
      return (await File.ReadAllTextAsync(testCardFile, ct)).Trim();
   }
}

/// <summary>Реальный считыватель на COM-порту: команда READ, ответ — UID карты
строкой.</summary>
public sealed class SerialCardReader(string portName, int baudRate) : ICardReader
{
   public string Description => $"{portName}, {baudRate} бод";

   public Task<string> ReadCardAsync(TimeSpan timeout, CancellationToken ct) => Task.Run(() =>
   {
      using var port = new System.IO.Ports.SerialPort(portName, baudRate)
      {
```

```
            ReadTimeout = (int)timeout.TotalMilliseconds,
            WriteTimeout = (int)timeout.TotalMilliseconds,
            NewLine = "\r\n"
        };
        port.Open();                        // UnauthorizedAccessException — порт занят другой программой
        port.WriteLine("READ");
        return port.ReadLine().Trim();      // TimeoutException — считыватель не ответил
    }, ct);
}

/// <summary>Проверка считывателя: чтение тестовой карты и проверка формата UID (4 или 7 байт).</summary>
public sealed partial class ReaderCheck(ICardReader reader) : IHealthCheck
{
    [GeneratedRegex("^([0-9A-F]{8}|[0-9A-F]{14})$")]
    private static partial Regex UidFormat();

    public string Code => "READER";
    public string Name => "Считыватель карт";
    public bool Critical => true;
    public int WarnLatencyMs(HealthCheckOptions o) => o.ReaderWarnMs;

    public async Task<string> RunAsync(HealthContext ctx, CancellationToken ct)
    {
        var uid = await reader.ReadCardAsync(TimeSpan.FromMilliseconds(ctx.Options.HealthCheck.Reader.TimeoutMs), ct);
        if (!UidFormat().IsMatch(uid))
            throw new InvalidDataException($"некорректный UID карты: '{uid}'");
        return $"{reader.Description}: прочитана тестовая карта UID {uid[..4]}****";
    }

    /// <summary>Выбор реализации по настройке Reader.Mode.</summary>
    public static ICardReader Create(ReaderOptions o, string emulatorCardFile) =>
        o.Mode.Equals("Serial", StringComparison.OrdinalIgnoreCase)
            ? new SerialCardReader(o.PortName, o.BaudRate)
            : new EmulatedCardReader(emulatorCardFile);
}
```

Листинг А.10 – Файл Campus.Arm.Updater/Reporting/ReadinessAct.cs

```
using System.Globalization;
using System.Text;
using Campus.Arm.Updater.Health;
using Campus.Arm.Updater.Infrastructure;

namespace Campus.Arm.Updater.Reporting;

/// <summary>Данные для акта готовности АРМ.</summary>
public sealed class ReadinessReport
{
    public required string ArmId { get; init; }
    public required string Operation { get; init; }
    public string VersionBefore { get; init; } = "—";
    public string VersionAfter { get; init; } = "—";
    public string ActiveSlot { get; init; } = "—";
    public string Ring { get; init; } = "—";
    public string? BackupPath { get; init; }
    public required DateTime StartedAt { get; init; }
    public DateTime FinishedAt { get; init; } = DateTime.Now;
    public required IReadOnlyList<HealthCheckResult> Results { get; init; }
    public List<string> Notes { get; init; } = [];
```

```
    /// <summary>АРМ готов, если не провалена ни одна критичная проверка и не было отката.</summary>
    public bool RolledBack { get; init; }
    public bool Ready => !RolledBack && Results.Count > 0 && Results.All(r => r.Status != HealthStatus.Fail || !
r.Critical);
}

/// <summary>
/// Формирование акта готовности АРМ к промышленной эксплуатации (Markdown) —
/// документ остаётся в каталоге reports/ и прикладывается к заявке на изменение.
/// </summary>
public static class ReadinessAct
{
    private static readonly CultureInfo Ru = CultureInfo.GetCultureInfo("ru-RU");

    public static string Write(ReadinessReport r, string reportsDir)
    {
        Directory.CreateDirectory(reportsDir);
        var file = Path.Combine(reportsDir, $"ACT_{r.ArmId}_{r.FinishedAt:yyyyMMdd_HHmmss_fff}.md");

        var sb = new StringBuilder();
        sb.AppendLine("# АКТ готовности АРМ к промышленной эксплуатации");
        sb.AppendLine();
        sb.AppendLine("ИС «Умный кампус» · клиентский модуль «АРМ оператора»");
        sb.AppendLine();
        sb.AppendLine("| Параметр | Значение |");
        sb.AppendLine("|---|---|");
        sb.AppendLine($"| АРМ | {r.ArmId} ({Environment.MachineName}) |");
        sb.AppendLine($"| Операция | {r.Operation} |");
        sb.AppendLine($"| Версия до / после | {r.VersionBefore} / {r.VersionAfter} |");
        sb.AppendLine($"| Активный слот (Blue-Green) | {r.ActiveSlot} |");
        sb.AppendLine($"| Кольцо развёртывания | {r.Ring} |");
        sb.AppendLine($"| Резервная копия | {(r.BackupPath is null ? "не создавалась" : Paths.Rel(r.BackupPath))}
|");
        sb.AppendLine($"| Начало / окончание | {r.StartedAt:dd.MM.yyyy HH:mm:ss} / {r.FinishedAt:dd.MM.yyyy
HH:mm:ss} |");
        sb.AppendLine();

        sb.AppendLine("## Результаты проверок (Health Check)");
        sb.AppendLine();
        sb.AppendLine("| № | Проверка | Критичная | Статус | Задержка, мс | Результат |");
        sb.AppendLine("|---|---|---|---|---|---|");
        var i = 1;
        foreach (var x in r.Results)
            sb.AppendLine($"| {i++} | {x.Name} | {(x.Critical ? "да" : "нет")} | {x.StatusText} |
{x.LatencyMs.ToString("0.0", Ru)} | {x.Details.Replace("|", "/")} |");
        sb.AppendLine();

        if (r.Results.Count > 0)
        {
            var lat = r.Results.Select(x => x.LatencyMs).ToList();
            sb.AppendLine($"Задержка проверок: мин. {lat.Min().ToString("0.0", Ru)} мс, " +
                          $"средн. {lat.Average().ToString("0.0", Ru)} мс, макс. {lat.Max().ToString("0.0", Ru)} мс, " +
                          $"суммарно {lat.Sum().ToString("0.0", Ru)} мс.");
            sb.AppendLine();
        }

        if (r.Notes.Count > 0)
        {
            sb.AppendLine("## Ход выполнения");
            sb.AppendLine();
            foreach (var n in r.Notes) sb.AppendLine($"- {n}");
            sb.AppendLine();
        }
```

```
        sb.AppendLine("## Заключение");
        sb.AppendLine();
        sb.AppendLine(r.Ready
            ? $"**АРМ {r.ArmId} ГОТОВ к промышленной эксплуатации** с версией {r.VersionAfter}."
            : r.RolledBack
                ? $"**АРМ {r.ArmId} НЕ ГОТОВ к эксплуатации версии {r.VersionAfter}.** Выполнен откат на версию {r.VersionBefore}; работа АРМ продолжается на предыдущей версии."
                : $"**АРМ {r.ArmId} НЕ ГОТОВ к промышленной эксплуатации.** Требуется устранение замечаний.");
        sb.AppendLine();
        sb.AppendLine("Системный администратор: _____  Оператор АРМ: _____");

        File.WriteAllText(file, sb.ToString(), new UTF8Encoding(encoderShouldEmitUTF8Identifier: true));
        return file;
    }

    /// <summary>Сводная таблица в консоль.</summary>
    public static void PrintSummary(ReadinessReport r)
    {
        Console.WriteLine();
        Console.WriteLine(new string('═', 78));
        Console.WriteLine($" АКТ ГОТОВНОСТИ АРМ {r.ArmId} · {r.Operation}");
        Console.WriteLine(new string('─', 78));
        Console.WriteLine($" {"Проверка",-46} {"Статус",-15} {"Задержка, мс",12}");
        foreach (var x in r.Results)
        {
            var old = Console.ForegroundColor;
            Console.ForegroundColor = x.Status switch
            {
                HealthStatus.Pass => ConsoleColor.Green,
                HealthStatus.Warn => ConsoleColor.Yellow,
                _ => ConsoleColor.Red
            };
            Console.WriteLine($" {x.Name,-46} {x.StatusText,-15} {x.LatencyMs,12:0.0}");
            Console.ForegroundColor = old;
        }
        Console.WriteLine(new string('─', 78));
        var c = Console.ForegroundColor;
        Console.ForegroundColor = r.Ready ? ConsoleColor.Green : ConsoleColor.Red;
        Console.WriteLine(r.Ready
            ? $" ИТОГ: АРМ ГОТОВ к промышленной эксплуатации (версия {r.VersionAfter})"
            : r.RolledBack
                ? $" ИТОГ: АРМ НЕ ГОТОВ к версии {r.VersionAfter} — выполнен ОТКАТ на {r.VersionBefore}"
                : " ИТОГ: АРМ НЕ ГОТОВ — см. акт и журнал");
        Console.ForegroundColor = c;
        Console.WriteLine(new string('═', 78));
    }
}
```

Листинг А.11 – Файл Campus.Arm.Updater/Infrastructure/AppLogger.cs

```
namespace Campus.Arm.Updater.Infrastructure;

/// <summary>
/// Журнал утилиты: цветной вывод в консоль + файл logs/updater-ГГГГММДД.log.
/// Ошибка записи в файл (диск заполнен, нет прав) не прерывает обновление —
/// сообщение остаётся в консоли.
/// </summary>
public sealed class AppLogger
{
```

```
    private readonly object _gate = new();
    private string? _file;

    /// <summary>Не писать в консоль (для автотестов).</summary>
    public bool Quiet { get; init; }

    public string? FilePath => _file;

    public AppLogger(string? logsDir)
    {
        if (logsDir is null) return;
        try
        {
            Directory.CreateDirectory(logsDir);
            _file = Path.Combine(logsDir, $"updater-{DateTime.Now:yyyyMMdd}.log");
        }
        catch (Exception ex) when (ex is IOException or UnauthorizedAccessException)
        {
            Console.Error.WriteLine($"[WRN] Каталог журнала недоступен ({ex.Message}); журнал только в
консоли.");
        }
    }

    public void Info(string message) => Write("INF", message, ConsoleColor.Gray);
    public void Step(string message) => Write("INF", message, ConsoleColor.Cyan);
    public void Ok(string message) => Write("INF", message, ConsoleColor.Green);
    public void Warn(string message) => Write("WRN", message, ConsoleColor.Yellow);
    public void Error(string message) => Write("ERR", message, ConsoleColor.Red);

    private void Write(string level, string message, ConsoleColor color)
    {
        var now = DateTime.Now;
        lock (_gate)
        {
            if (!Quiet)
            {
                var old = Console.ForegroundColor;
                Console.ForegroundColor = color;
                Console.WriteLine($"{now:HH:mm:ss.fff} [{level}] {message}");
                Console.ForegroundColor = old;
            }

            if (_file is null) return;
            try
            {
                File.AppendAllText(_file, $"{now:yyyy-MM-dd HH:mm:ss.fff} [{level}] {message}
{Environment.NewLine}");
            }
            catch (Exception ex) when (ex is IOException or UnauthorizedAccessException)
            {
                // Файл журнала занят или диск недоступен — дальше пишем только в консоль.
                _file = null;
                if (!Quiet) Console.Error.WriteLine($"[WRN] Запись в журнал прекращена: {ex.Message}");
            }
        }
    }
}
```

Листинг А.12 – Файл Campus.Arm.Updater/Infrastructure/Common.cs

```
using System.Security.Cryptography;
using System.Text.Encodings.Web;
```

```
using System.Text.Json;
using System.Text.Unicode;

namespace Campus.Arm.Updater.Infrastructure;

/// <summary>Общие настройки JSON (camelCase, комментарии и кириллица без
экранирования).</summary>
public static class Json
{
   public static readonly JsonSerializerOptions Options = new(JsonSerializerDefaults.Web)
   {
      WriteIndented = true,
      ReadCommentHandling = JsonCommentHandling.Skip,
      AllowTrailingCommas = true,
      Encoder = JavaScriptEncoder.Create(UnicodeRanges.All)
   };
}

/// <summary>Контрольные суммы для проверки целостности пакетов, файлов и резервных
копий.</summary>
public static class Hashing
{
   public static string Sha256File(string path)
   {
      using var stream = File.OpenRead(path);
      return Convert.ToHexString(SHA256.HashData(stream)).ToLowerInvariant();
   }
}

/// <summary>Повтор файловых операций при временных ошибках (файл занят антивирусом, сетевая папка
«моргнула»).</summary>
public static class Retry
{
   public static T Io<T>(Func<T> action, AppLogger log, string what, int attempts = 3)
   {
      for (var i = 1; ; i++)
      {
         try
         {
            return action();
         }
         catch (IOException ex) when (i < attempts)
         {
            log.Warn($"{what}: ошибка ввода-вывода ({ex.Message}), повтор {i}/{attempts - 1} через {i * 500}
мс");
            Thread.Sleep(i * 500);
         }
      }
   }
}

/// <summary>Короткая запись путей в журнале: относительно текущего каталога, если это
возможно.</summary>
public static class Paths
{
   public static string Rel(string path)
   {
      var relative = Path.GetRelativePath(Environment.CurrentDirectory, path);
      return relative.StartsWith("..") || Path.IsPathRooted(relative) ? path : relative;
   }
}
```

```
/// <summary>Коды завершения утилиты (используются планировщиком/скриптами
развёртывания).</summary>
public static class ExitCodes
{
    public const int Ok = 0;            // АРМ готов / операция выполнена
    public const int NotReady = 1;      // проверки не пройдены
    public const int RolledBack = 2;    // обновление откачено
    public const int Aborted = 3;       // обновление прервано до установки (пакет повреждён, сервер недоступен)
    public const int ConfigError = 4;   // ошибка настроек или аргументов
}
```

Листинг А.13 – Файл Campus.Arm.Updater/Infrastructure/LocalDb.cs

```
using System.Text.RegularExpressions;
using Microsoft.Data.Sqlite;

namespace Campus.Arm.Updater.Infrastructure;

/// <summary>Работа с локальной БД АРМ (SQLite). Пул соединений выключен, чтобы файл БД
/// не оставался заблокированным во время резервного копирования и отката.</summary>
public static partial class LocalDb
{
    [GeneratedRegex("^[A-Za-z_][A-Za-z0-9_]*$")]
    private static partial Regex Identifier();

    public static SqliteConnection Open(string path, bool readOnly = false)
    {
        if (!File.Exists(path))
            throw new FileNotFoundException("Локальная БД АРМ не найдена", path);

        var cs = new SqliteConnectionStringBuilder
        {
            DataSource = path,
            Mode = readOnly ? SqliteOpenMode.ReadOnly : SqliteOpenMode.ReadWrite,
            Pooling = false
        }.ToString();

        var connection = new SqliteConnection(cs);
        connection.Open();
        return connection;
    }

    public static object? Scalar(SqliteConnection c, string sql)
    {
        using var cmd = c.CreateCommand();
        cmd.CommandText = sql;
        return cmd.ExecuteScalar();
    }

    public static int GetSchemaVersion(SqliteConnection c) =>
        Convert.ToInt32(Scalar(c, "SELECT version FROM schema_info LIMIT 1"));

    public static bool ColumnExists(SqliteConnection c, string table, string column)
    {
        EnsureIdentifier(table);
        using var cmd = c.CreateCommand();
        cmd.CommandText = $"SELECT COUNT(*) FROM pragma_table_info('{table}') WHERE name = $column";
        cmd.Parameters.AddWithValue("$column", column);
        return Convert.ToInt64(cmd.ExecuteScalar()) > 0;
    }
```

```
    /// <summary>Количество строк в ключевых таблицах — «снимок» для проверки, что данные не потерялись.</summary>
    public static Dictionary<string, long> CountRows(SqliteConnection c, params string[] tables)
    {
        var result = new Dictionary<string, long>();
        foreach (var table in tables)
        {
            EnsureIdentifier(table);
            result[table] = Convert.ToInt64(Scalar(c, $"SELECT COUNT(*) FROM {table}"));
        }
        return result;
    }

    public static readonly string[] KeyTables = ["access_events", "operators", "sync_queue"];

    private static void EnsureIdentifier(string name)
    {
        if (!Identifier().IsMatch(name))
            throw new ArgumentException($"Недопустимое имя таблицы: {name}");
    }
}
```

Листинг А.14 – Файл tests/Campus.Arm.Updater.Tests/SanityTests.cs

```
using System.Net;
using System.Text;
using Campus.Arm.Updater.Config;
using Campus.Arm.Updater.Demo;
using Campus.Arm.Updater.Health;
using Campus.Arm.Updater.Infrastructure;
using Campus.Arm.Updater.Update;

namespace Campus.Arm.Updater.Tests;

/// <summary>
/// Этап 4. Sanity-тесты: имитация установки обновления на стенде во временном каталоге,
/// проверка целостности данных, откат и формирование акта готовности.
/// Внешний API заменён заглушкой, поэтому тесты не зависят от интернета.
/// </summary>
public sealed class SanityTests : IDisposable
{
    private readonly string _dir = Path.Combine(Path.GetTempPath(), $"arm-sanity-{Guid.NewGuid():N}");
    private readonly UpdaterOptions _options;
    private readonly AppLogger _log;

    public SanityTests()
    {
        Directory.CreateDirectory(_dir);
        var config = DemoEnvironment.Create(_dir);
        _options = UpdaterOptions.Load(config);
        _options.HealthCheck.ApiTimeoutSeconds = 1;
        _options.HealthCheck.ApiRetries = 2;
        _log = new AppLogger(_options.LogsRoot) { Quiet = true };
    }

    private ArmLayout Layout => new(_options.InstallRoot);

    /// <summary>Фабрика создаёт новый обработчик на каждую проверку (HttpClient освобождает его после запроса).</summary>
    private UpdateOrchestrator Orchestrator(Func<HttpMessageHandler>? api = null) =>
        new(_options, _log, api ?? (() => new StubApi(HttpStatusCode.OK, """{"ip":"95.24.13.7"}""")));
```

```
    [Fact]
    public async Task Update_GoodRelease_SwitchesSlot_KeepsData_AndActSaysReady()
    {
        var code = await Orchestrator().UpdateAsync("2.5.0");

        Assert.Equal(ExitCodes.Ok, code);
        Assert.Equal("green", Layout.ReadActiveSlot());
        Assert.Equal("2.5.0", Layout.ReadVersion("green")!.Version);

        using var db = LocalDb.Open(Layout.Database, readOnly: true);
        Assert.Equal(2, LocalDb.GetSchemaVersion(db));
        Assert.True(LocalDb.ColumnExists(db, "access_events", "reader_id"));
        Assert.Equal(1200, LocalDb.CountRows(db, "access_events")["access_events"]);   // данные не потеряны

        var act = await File.ReadAllTextAsync(Directory.GetFiles(_options.ReportsRoot, "ACT_*.md").Single());
        Assert.Contains("ГОТОВ к промышленной эксплуатации", act);
        Assert.Single(Directory.GetDirectories(_options.BackupRoot));                  // резервная копия создана
    }

    [Fact]
    public async Task Update_BrokenMigration_RollsBack_RestoresDatabase()
    {
        Assert.Equal(ExitCodes.Ok, await Orchestrator().UpdateAsync("2.5.0"));

        var code = await Orchestrator().UpdateAsync("2.5.1");

        Assert.Equal(ExitCodes.RolledBack, code);
        Assert.Equal("green", Layout.ReadActiveSlot());                                // снова 2.5.0
        Assert.Equal("2.5.0", Layout.ReadVersion(Layout.ReadActiveSlot())!.Version);

        using var db = LocalDb.Open(Layout.Database, readOnly: true);
        Assert.Equal(2, LocalDb.GetSchemaVersion(db));                                 // схема восстановлена из копии
        Assert.False(LocalDb.ColumnExists(db, "access_events", "readr_zone"));
        Assert.Equal(1200, LocalDb.CountRows(db, "access_events")["access_events"]);
    }

    [Fact]
    public async Task ManualRollback_AfterFailedUpdate_ReturnsToPreviousGoodVersion()
    {
        await Orchestrator().UpdateAsync("2.5.0");     // 2.4.0 → 2.5.0 (green)
        await Orchestrator().UpdateAsync("2.5.1");     // сбой, автооткат, слот blue очищен
        Assert.Null(Layout.ReadVersion("blue"));

        var code = await Orchestrator().RollbackAsync(); // ручной откат 2.5.0 → 2.4.0

        Assert.Equal(ExitCodes.Ok, code);
        Assert.Equal("blue", Layout.ReadActiveSlot());
        Assert.Equal("2.4.0", Layout.ReadVersion("blue")!.Version);   // переустановлена с сервера
        using var db = LocalDb.Open(Layout.Database, readOnly: true);
        Assert.Equal(1, LocalDb.GetSchemaVersion(db));                 // данные из копии версии 2.4.0
        Assert.Equal(1200, LocalDb.CountRows(db, "access_events")["access_events"]);
    }

    [Fact]
    public async Task Update_CorruptedPackage_IsRejected_BeforeAnyChange()
    {
        var code = await Orchestrator().UpdateAsync("2.6.0");

        Assert.Equal(ExitCodes.Aborted, code);
        Assert.Equal("blue", Layout.ReadActiveSlot());
        Assert.False(Directory.Exists(_options.BackupRoot) &&
Directory.GetDirectories(_options.BackupRoot).Length > 0);
    }
```

```
    [Fact]
    public async Task HealthCheck_ApiTimeout_IsHandled_AndArmIsNotReady()
    {
        var code = await Orchestrator(() => new StubApi(delay: TimeSpan.FromSeconds(10))).HealthCheckAsync();

        Assert.Equal(ExitCodes.NotReady, code);
        var act = await File.ReadAllTextAsync(Directory.GetFiles(_options.ReportsRoot, "ACT_*.md").Single());
        Assert.Contains("таймаут", act);
        Assert.Contains("НЕ ГОТОВ", act);
    }

    [Fact]
    public async Task Update_ApiDownAfterSwitch_TriggersRollback()
    {
        var code = await Orchestrator(() => new StubApi(HttpStatusCode.ServiceUnavailable,
"")).UpdateAsync("2.5.0");

        Assert.Equal(ExitCodes.RolledBack, code);
        Assert.Equal("blue", Layout.ReadActiveSlot());
        using var db = LocalDb.Open(Layout.Database, readOnly: true);
        Assert.Equal(1, LocalDb.GetSchemaVersion(db));
    }

    [Fact]
    public async Task HealthCheck_MissingReaderCard_FailsReaderOnly()
    {
        File.Delete(Layout.ReaderTestCard);
        var checks = new IHealthCheck[] { new DatabaseCheck(), new ReaderCheck(new
EmulatedCardReader(Layout.ReaderTestCard)) };

        var results = await new HealthCheckRunner(checks).RunAsync(new HealthContext { Layout = Layout,
Options = _options, Log = _log });

        Assert.Equal(HealthStatus.Pass, results[0].Status);
        Assert.Equal(HealthStatus.Fail, results[1].Status);
        Assert.All(results, r => Assert.True(r.LatencyMs >= 0));
    }

    [Fact]
    public void ExternalIp_IsMasked() => Assert.Equal("95.24.*.*", ApiCheck.Mask("95.24.13.7"));

    public void Dispose()
    {
        Microsoft.Data.Sqlite.SqliteConnection.ClearAllPools();
        try { Directory.Delete(_dir, recursive: true); } catch (IOException) { }
    }

    /// <summary>Заглушка внешнего API: заданный ответ или задержка (имитация сетевого
таймаута).</summary>
    private sealed class StubApi(HttpStatusCode status = HttpStatusCode.OK, string body = "", TimeSpan? delay =
null) : HttpMessageHandler
    {
        protected override async Task<HttpResponseMessage> SendAsync(HttpRequestMessage request,
CancellationToken ct)
        {
            if (delay is not null) await Task.Delay(delay.Value, ct);
            return new HttpResponseMessage(status) { Content = new StringContent(body, Encoding.UTF8,
"application/json") };
        }
    }
}
```